Dotter: Mathematical Analysis and Implementation Guide

Oscar Laird

March 2026

Contents

Introduction

Dotter is a text-entry technology that allows users to type over a 1-bit channel. It allows a simple input mechanism such as blinking or pressing a single button to be used to type. Dotter milks as much information out of that 1-bit channel as possible, allowing an experienced user to type standard english text at speeds of 20 words per minute only by blinking.

> **Dotter only has one rule:** Identify the longest visible prefix of your target text and synchronize your signal to its timer.

Dotter was inspired by the project "Dasher" [2], a text-entry technology developed by David MacKay's group at the University of Cambridge that took inspiration from arithmetic coding to map the space of all possible text sequences to the unit interval. Assuming a version of Fitt's law, Dasher is an optimal text-entry technology for 1-dimensional continuous (pointing) input.

This document is a mathematical analysis of Dotter as well as an implementation guide, describing the architecture and key algorithms of Dotter. Chapter 1 describes the architecture of Dotter, outlining its key data structures and algorithms. The succeeding chapters describe individual components of Dotter. The final chapter gives theory and advice on how to learn to type with Dotter. (If the person reading this only wants to learn how to use Dotter, they should skip to the final chapter.)

The name "Dotter" is a nod the most famous 1-bit text-entry system, telegraphy, which consisted of "dots" and "dashes." The name "Dasher" was already taken, so I went with this.

Chapter 1

Architecture

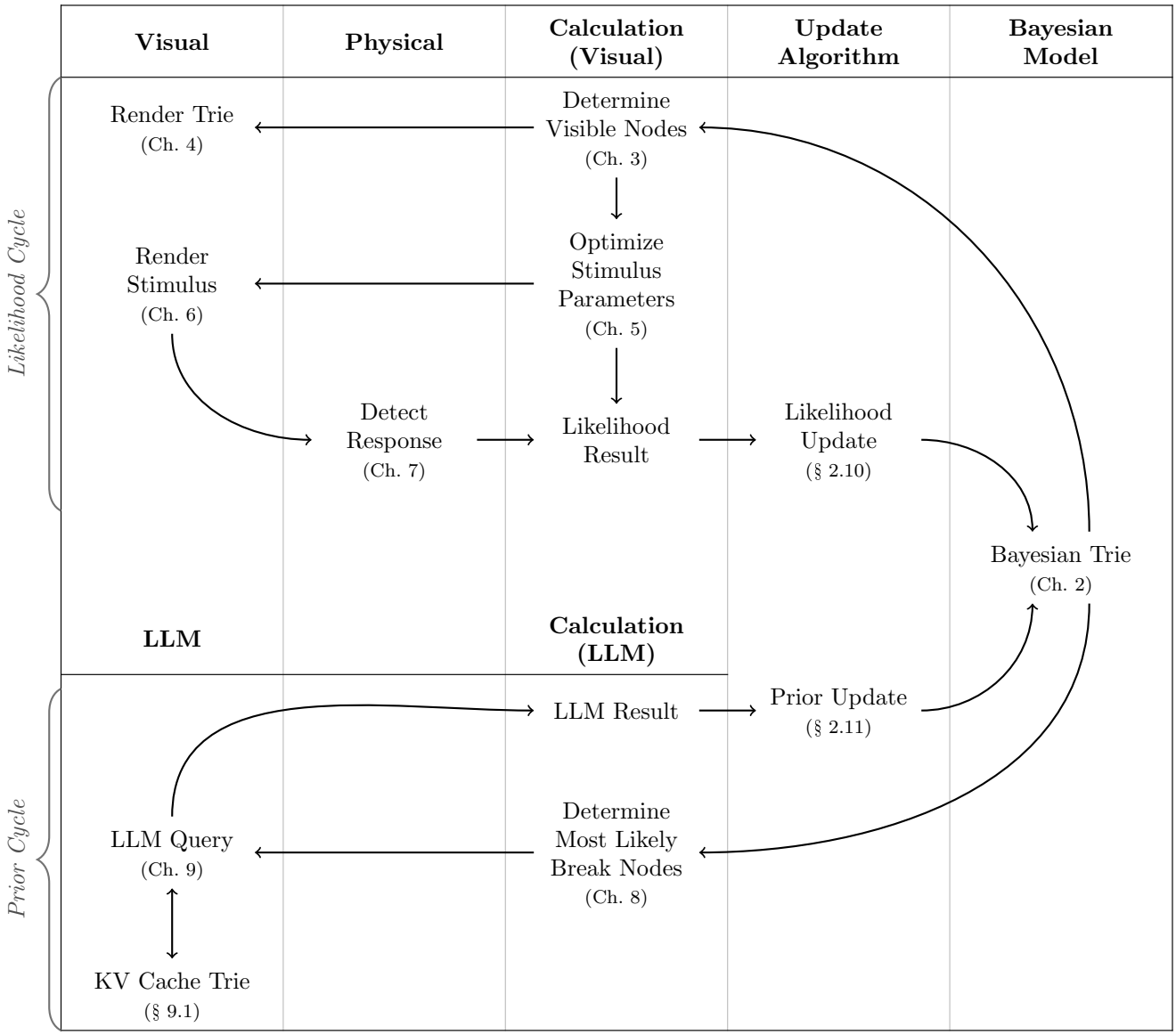

Table 1.1: Architecture of Dotter

Table 1.1 shows the architecture of Dotter. At its heart is the "bayesian trie" (chapter 2) in the rightmost column, which is a character-based Bayesian prefix trie which maintains a calculation of the posterior probability of every prefix. The other elements of the system may be separated into the Likelihood Cycle and the Prior Cycle.

Each element in Table 1.1 is discussed in detail in its own chapter or section.

1.1 The Likelihood Cycle

The Likelihood Cycle refers to the series of steps by which we elicit a signal from the user and then interpret that signal to update our posterior probabilities. It has seven steps,

1. **Determine Visible Nodes** (chapter 3): we determine which prefixes should be visually displayed by checking which prefixes have a posterior probability exceeding the visibility threshold (e.g. 2%).
2. **Render Trie** (chapter 4): here we draw the visible prefixes to the screen, connecting parent nodes to their children with smooth curves.
3. **Optimize Stimulus Parameters** (chapter 5): in this step we assign timers to every prefix. We want to maximize the information gain from the user's signal which is accomplished by making the timers as far apart from each other as possible (in a formal sense). This helps to make the user's selection unambiguous.
4. **Render Stimulus** (chapter 6): we draw the timers to the screen.
5. **Detect Response** (chapter 7): we detect the user's signal. It is trivial to detect a keypress; detecting the exact timing of a blink is a more sophisticated problem.
6. **Likelihood Result**: we compare the detected user signal to the timers that were assigned in step 3. This gives us a likelihood score for each prefix.
7. **Likelihood Update** (§ 2.10): we update the trie to reflect the received likelihoods. Performing this update in a lazy and efficient manner is a central technical challenge of Dotter. (This is an "update" in the traditional bayesian sense.)

1.2 The Prior Cycle

The Prior Cycle refers to the series of steps by which we query a large language model to refine our prior probabilities and incorporate those results into our posterior beliefs. It has four steps,

1. **Determine Most Likely Break Nodes** (chapter 8): we determine which nodes are most likely to a break between tokens in the tokenization of the user's target text.
2. **LLM Query** (chapter 9): we query the large language model to predict the distribution over the next token for the prefixes identified in the previous step, making use of the Key-Value Cache Trie (§ 9.1).
3. **LLM Result**: we receive the result from the language model.
4. **Prior Update** (§ 2.11): we update the trie to reflect the received prior probabilities. Performing this update in a lazy and efficient manner is a central technical challenge of Dotter. (This is not truly an "update" in the traditional bayesian sense, since normally our priors are fixed.)

1.3 Client-Server Communication

A web-based client is responsible for the likelihood cycle, while a server with a large language model handles the prior cycle. Since these two cycles must run independently, it is necessary for both the client and the server to maintain a copy of the trie.

The data designated "Likelihood Result" and "LLM Result" are exchanged between the client and the server, with each side updating its copy of the trie in response to the stream of these results.

The two sides will not necessarily receive this stream of results in the same order, but an important correctness property of our update algorithms is that the state of the trie does not depend on the order in which the updates are performed.

Chapter 2

The Bayesian Trie

Dotter must determine the most likely string that the user is typing, based on observation of the user's signals. To accomplish this, Dotter maintains a trie indicating the posterior probability that any given prefix is a prefix of the user's target string.

To compare strings, we use the inequality notation $X \leq Y$ to mean that the string X is a prefix of the string Y.

Letting X be the user's target string, D be the user's signals, then the posterior probability that a given string, S, is a prefix of X is given by:

$$P(S \leq X|D) = \frac{P(D|S \leq X)P(S \leq X)}{P(D)}$$

In practice, we only keep track of the unnormalized posterior probabilities, denoted in code as `post_Z` (to mean the posterior probability multiplied by a normalizing constant). Since the root node, S_0, is the empty string, we have $P(S_0 \leq X) = 1$ and thus the unnormalized posterior probability of the root node is $P(D)$, i.e., the normalizing constant. Thus if we know the unnormalized posterior probabilities in our trie, we can convert these to normalized posterior probabilities by dividing by the root node's unnormalized posterior probability.

The posterior probabilities are derived from the prior probabilities and the likelihoods. In Dotter, both are subject to updates. The likelihood is updated as we receive signals from the user and the prior is updated as our language model refines our predictions.

Our analysis has five parts:

1. We assume a character-based prior model, present algorithms for lazy calculation of the unnormalized posteriors, and prove their correctness.
2. We discuss implementation details with regard to tracking the likelihoods.
3. We discuss converting a token-based prior model to a character-based prior model.
4. We discuss how to calculate the branch prior for a node.
5. We discuss how to track the maximum descendant likelihoods of token sequences.
6. We present the full algorithms.

2.1 Lazy Posterior Calculation: The Core Algorithms

Definition 2.1 (Alphabet)**.** *An alphabet is a finite set.*

Definition 2.2 (Character)**.** *A character is an element of an alphabet.*

Definition 2.3 (String)**.** *A string is a sequence of characters.*

Definition 2.4 (Substring)**.** *We use the notation $s[i : j]$ to represent the subsequence of characters of s from index i to index $j - 1$. We also allow the notation $s[: -k]$ to represent the subsequence of characters of s from index 0 to index $|s| - k - 1$.*

Definition 2.5 (String Inequality)**.** *We say that a string s is a prefix of a string t, and write this as $s \leq t$, if and only if there exists an integer k such that $s = t[0:k]$.*

Definition 2.6 (Properly Terminated String)**.** *A string h is properly terminated if and only if $h[i] = \text{'■'} \iff i = |h| - 1$. An unterminated string is one that does not contain the stop character. An "at most properly terminated string" is one that is either properly terminated or unterminated.*

Throughout this chapter, we will let $\mathcal{A}$ denote our alphabet which must include at least the two special characters: space (' ') and stop ('■'); the set of all strings over this alphabet will be denoted S, and the subset of all properly terminated strings will be denoted H, where if $h \in H$.

Definition 2.7 (Prefix Code)**.** *A prefix code is a set of strings such that no string is a prefix of another string in the set, i.e., A set C is a prefix code if and only if,*

$$\nexists c_1, c_2 \in C, c_1 < c_2$$

Definition 2.8 (Complete Prefix Code)**.** *A complete prefix code, C, is a prefix code that covers the entire tree, i.e.,*

$$\forall x \in S, \exists c \in C, x \leq c \vee c \leq x$$

Definition 2.9 (Refinement)**.** *A prefix code, A, is a refinement of a prefix code, B, if every element of A is an extension of some element of B, i.e.,*

$$\forall a \in A, \exists b \in B, a \geq b$$

If this is the case, we write $A \geq B$.

Refinement is a partial order over complete prefix codes. These prefix codes form a lattice, so we may define the meet and join of two prefix codes.

Definition 2.10 (Meet)**.** *The meet of two prefix codes, A and B, is the greatest lower bound of A and B, i.e., i.e., it is the set of minimal elements of their intersection. We write this as $A \wedge B$.*

Definition 2.11 (Join)**.** *The join of two prefix codes, A and B, is the least upper bound of A and B, i.e., it is the set of maximal elements of their union. We write this as $A \vee B$.*

Definition 2.12 (Element Refinement)**.** *We overload the comparison operators when the left hand side is a string and the right hand side is a complete prefix code. $x \leq C$ means that x is a prefix of some element of C, whereas $x > C$ means that x is a proper extension of some element of C.*

Proposition 2.1. *The set of all properly terminated strings, H, is a complete prefix code.*

Proof. We first show that H is a prefix code. We proceed by contradiction. Suppose there exist $h_1, h_2 \in H$ such that $h_1 < h_2$. Because h_1 is a strict prefix of h_2, it must be that $|h_1| < |h_2|$. Furthermore, the characters of h_2 must perfectly match h_1 up to index $|h_1| - 1$. Thus,

$$h_2[|h_1| - 1] = h_1[|h_1| - 1].$$

Because $h_1 \in H$, it is properly terminated, meaning its final character is the stop character: $h_1[|h_1|-1] = \text{'■'}$. Therefore, $h_2[|h_1|-1] = \text{'■'}$. However, because $h_2 \in H$, the stop character can only appear at its final index $|h_2| - 1$. This implies:

$$|h_1| - 1 = |h_2| - 1 \implies |h_1| = |h_2|,$$

which contradicts the strict prefix requirement $|h_1| < |h_2|$. Thus, no such pair h_1, h_2 can exist, and H is a prefix code.

We now show that H is a complete prefix code. Let $x \in S$ be any string. We show this by cases constructively:

(a) If '■' is in x, then let i be the index of its first occurrence. Then $x[:i+1] \in H$ and $x[:i+1] \leq x$.

(b) If '■' is not in x, then $x + \text{'■'} \in H$ and $x < x + \text{'■'}$.

□

Definition 2.13 (Likelihood Function)**.** *A likelihood function is a mapping from properly terminated strings to non-negative real numbers, $L_n : H \to \mathbb{R}^+$.*

Definition 2.14 (Prior Function)**.** *A prior function is a mapping from properly terminated strings to non-negative real numbers, $P_m : H \to \mathbb{R}^+$, satisfying*

$$\sum_{h \in H} P_m(h) = 1$$

Definition 2.15 (Branch Prior Function)**.** *We define the branch prior function, $P_m^{br}(x)$, of an at most properly terminated string x, as,*

$$P_m^{br}(x) = \sum_{\substack{h \in H \\ h \geq x}} P_m(h)$$

Terminology Note: The branch prior function corresponds to our usual notion of a prior over string prefixes, but we choose to name it explicitly as "branch prior" and to restrict our definition of the domain of a "prior function" to properly terminated strings, excluding their prefixes. We make this choice to keep a close correspondence between the definitions of prior and likelihood functions.

Note: In our algorithm, we will assume oracular access to the branch prior function.

Definition 2.16 (Likelihood Sequence)**.** *A likelihood sequence is a sequence of likelihood functions, $\{L_n\}$ where $\forall h, L_0(h) = 1$.*

Definition 2.17 (Prior Sequence)**.** *A prior sequence is a sequence of prior functions $\{P_n\}$.*

Definition 2.18 (Delta Digest)**.** *A delta digest is a prefix code and a mapping from that prefix code to non-negative real numbers, (C, Δ). So,*

$$\Delta : C \to \mathbb{R}^+$$

Definition 2.19 (Prefix Code Truncation)**.** *The truncation of a string x under a prefix code C refers to the element $c \in C$ such that $c \leq x$. If no such element exists, then the truncation is undefined. We denote the truncation of x under C as $\lfloor x \rfloor_C$.*

Definition 2.20 (Likelihood Delta Digest Sequence)**.** *The likelihood delta digest sequence corresponding to a likelihood sequence $\{L_n\}$ is a sequence of delta digests $\{(C_n^L, \Delta_n^L)\}$, that satisfies:*

$$\forall h \in H, L_n(h) = L_{n-1}(h)\Delta_n^L(\lfloor h \rfloor_{C_n^L})$$

Definition 2.21 (Prior Delta Digest Sequence)**.** *This is defined identically, i.e., the prior delta digest sequence corresponding to a prior sequence $\{P_n\}$ is a sequence of delta digests $\{(C_n^P, \Delta_n^P)\}$, that satisfies:*

$$\forall h \in H, P_n(h) = P_{n-1}(h)\Delta_n^P(\lfloor h \rfloor_{C_n^P})$$

Lemma 2.1 (Eventually Constant Likelihoods)**.** *Given the cumulative likelihood update frontier, $\bigvee_{i=1}^N C_i^L$, and a given node in that frontier, f, the likelihoods of all properly terminated strings descended from that node are constant with respect to the branch up to time N, i.e.,*

$$\forall n \leq N, \forall h \in H, h \geq f, L_n(h) = \prod_{1 \leq i \leq n} \Delta_i^L(\lfloor f \rfloor_{C_i^L})$$

Proof. Recall that $L_0(h) = 1$. By definition of the likelihood sequence,

$$L_n(h) = \prod_{1 \leq i \leq n} \Delta_i^L(\lfloor h \rfloor_{C_i^L}).$$

We must show that $\lfloor h \rfloor_{C_i^L} = \lfloor f \rfloor_{C_i^L}$ for all $1 \leq i \leq n$.

Any element $f \in \bigvee_{i=1}^{N} C_i^L$ is a descendant of or equal to an element in each C_i^L. In other words, for every $i \in \{1, \ldots, n\}$, there exists some $c_i \in C_i^L$ such that $c_i \leq f$. Because c_i is a prefix of f, it is exactly the truncation of f under C_i^L, so $c_i = \lfloor f \rfloor_{C_i^L}$.

By hypothesis, $h \geq f$. Since the prefix relation is transitive, $c_i \leq f$ and $f \leq h$ imply $c_i \leq h$. Because C_i^L is a prefix code, c_i is the unique element in C_i^L that is a prefix of h. Therefore, $\lfloor h \rfloor_{C_i^L} = c_i = \lfloor f \rfloor_{C_i^L}$. Substituting this into the product yields the result. □

Definition 2.22 (Unnormalized Posterior Function)**.** *The unnormalized posterior function is a mapping from at most properly terminated strings to non-negative real numbers given by*

$$Z_{n,m}(x) = \sum_{\substack{h \in H \\ h \geq x}} L_n(h) P_m(h)$$

Proposition 2.2. *Without any likelihood updates, the unnormalized posterior of the root node is 1:*

$$\begin{aligned}
Z_{0,m}(\text{''}) &= \sum_{\substack{h \in H \\ h \geq \text{''}}} L_0(h) P_m(h) \\
&= \sum_{h \in H} L_0(h) P_m(h) \\
&= \sum_{h \in H} 1 \cdot P_m(h) && \textit{(by definition 2.16)} \\
&= 1 && \textit{(by definition 2.14)}
\end{aligned}$$

Lemma 2.2 (Posterior Equals Sum of Children)**.** *if x is an unterminated string, then*

$$Z_{n,m}(x) = \sum_{a \in \mathcal{A}} Z_{n,m}(x + a)$$

Proof. By definition,

$$Z_{n,m}(x) = \sum_{\substack{h \in H \\ h \geq x}} L_n(h) P_m(h).$$

Since x is an unterminated string, it does not contain the stop character. Because all strings in H are properly terminated and therefore end with the stop character, $x \notin H$. Thus, any properly terminated string $h \in H$ that has x as a prefix must be strictly longer than x ($|h| > |x|$).

Therefore, the character in h immediately following the prefix x must be some character $a \in \mathcal{A}$. We can partition the set of strings $\{h \in H \mid h \geq x\}$ into mutually disjoint subsets based on this next character a:

$$\{h \in H \mid h \geq x\} = \bigcup_{a \in \mathcal{A}} \{h \in H \mid h \geq x + a\}.$$

Because these subsets are disjoint, we can split the summation over them:

$$\begin{aligned}
Z_{n,m}(x) &= \sum_{a \in \mathcal{A}} \sum_{\substack{h \in H \\ h \geq x + a}} L_n(h) P_m(h) \\
&= \sum_{a \in \mathcal{A}} Z_{n,m}(x + a).
\end{aligned}$$

□

We may now introduce the datastructure at the core of Dotter, the Bayesian Trie. The Bayesian Trie maintains a calculation of the unnormalized posterior probability while allowing for the lazy application of likelihood and prior updates.

Definition 2.23 (Bayesian Trie Node)**.** *A Bayesian trie node x associated with a string $s \in S$ is a data structure containing the tuple (n, m, Z), accessed via fields:*

- $x.n \in \mathbb{N}$*, the "likelihood time" of the node.*
- $x.m \in \mathbb{N}$*, the "prior time" of the node.*
- $x.Z \in \mathbb{R}^+$*, a computed quantity which should be equal to the unnormalized posterior of the node evaluated at those times, i.e., $x.Z = Z_{x.n,x.m}(s)$.*

Definition 2.24 (Valid Node)**.** *A node, x, is called valid at time (N, M) if it satisfies one of the following two conditions:*

1. ***Constant Likelihood****: x lies beyond the cumulative likelihood update frontier, i.e., $x \geq \bigvee_{i=1}^{N} C_i^L$*
2. ***No Reweighting****: The likelihood and the prior have changed by a constant factor since the node's time, i.e., $x \geq \bigvee_{i=x.n+1}^{N} C_i^L$ and $x \geq \bigvee_{i=x.m+1}^{M} C_i^P$*

Lemma 2.3 (Current Nodes are Valid)**.** *A node is always valid at its own time. I.e., $(x.n, x.m) = (N, M)$ implies that x is valid at time (N, M).*

Proof. Such a node satisfies the "No Reweighting" condition of validity. Because $(x.n, x.m) = (N, M)$, the two joins are joins over empty sets and thus equal to the root node. Hence, $x \geq \bigvee_{i=x.n+1}^{N} C_i^L$ and $x \geq \bigvee_{i=x.m+1}^{M} C_i^P$. □

Definition 2.25 (Bayesian Trie)**.** *A Bayesian Trie is a trie of Bayesian trie nodes along with a global tuple describing the current time, (N, M).*

Definition 2.26 (Valid Bayesian Trie)**.** *A Bayesian Trie is called valid, if all of its nodes are valid for the trie's global current time, (N, M).*

Definition 2.27 (Node Correctness)**.** *A node is "correct" if it correctly calculates the unnormalized posterior,*

$$x.Z = Z_{x.n,x.m}(x)$$

Definition 2.28 (Bayesian Trie Correctness)**.** *The Bayesian trie is "correct" if all of its nodes are correct.*

We may now give the algorithm to advance a node's time. This algorithm recalculates the unnormalized posterior of a valid node, x, to a new time, (n', m'). Recall that we assume oracular access to the branch prior function, $P_m^{br}(x)$.

Algorithm 1 Advance Node Time

Require: x, the node to update.
Require: (n, m), the node's current time.
Require: (n', m'), the new time.
Require: (N, M), the trie's current time.
Require: x is valid at time (N, M).

if $x \geq \bigvee_{i=x.n+1}^{N} C_i^L$ and $x \geq \bigvee_{i=x.m+1}^{M} C_i^P$ **then**
    $x.Z \leftarrow x.Z \prod_{i=x.n+1}^{n'} \Delta_i^L(\lfloor x \rfloor_{C_i^L}) \prod_{i=x.m+1}^{m'} \Delta_i^P(\lfloor x \rfloor_{C_i^P})$
else $x \geq \bigvee_{i=1}^{N} C_i^L$
    $x.Z \leftarrow P_{m'}^{br}(x) \prod_{i=1}^{n'} \Delta_i^L(\lfloor x \rfloor_{C_i^L})$
$x.n \leftarrow n'$
$x.m \leftarrow m'$

We also provide for advancing the trie to a new global time.

Algorithm 2 Advance Trie Time

Require: (N, M), the trie's current time.
Require: *trie* is valid at time (N, M).
Require: (N', M'), the new time.
1: $trie.N \leftarrow N'$
2: $trie.M \leftarrow M'$
3: $F \leftarrow \bigvee_{i=1}^{N'} C_i^L \wedge \left(\bigvee_{i=N+1}^{N'} C_i^L \vee \bigvee_{i=M+1}^{M'} C_i^P\right)$ ▷ $x \geq F$ implies x is valid at time (N', M').
4: Run `AdvanceNodeTime` to update the time of each node in this frontier to (N', M').
5: **for** each node x strictly within F, $x < F$, proceeding in reverse topological order **do**
6: $\quad x.Z \leftarrow \sum_{y \in x.\text{children}} y.\text{Z}$
7: $\quad x.n \leftarrow N'$
8: $\quad x.m \leftarrow M'$

We now show that both algorithms preserve the validity of the trie and its correctness.

Theorem 2.1 (Advance Node Time Validity)**.** *The* `AdvanceNodeTime` *algorithm preserves the validity of the node.*

Proof. The "constant likelihood" depends only on the global time, so updating the node's time does not affect if this condition is satisfied. The "no reweighting" condition is weakened by increasing the node's time. I.e.,

$$x \geq \bigvee_{i=n+1}^{N} C_i^L \implies x \geq \bigvee_{i=n'+1}^{N} C_i^L$$

and,

$$x \geq \bigvee_{i=m+1}^{M} C_i^P \implies x \geq \bigvee_{i=m'+1}^{M} C_i^P$$

if $n' \geq n$. □

Theorem 2.2 (Advance Node Time Correctness)**.** *The* `AdvanceNodeTime` *algorithm preserves the correctness of the node.*

Proof. We show this for the two cases of valid nodes.

Constant Likelihood Case: If x satisfies, $x \geq \bigvee_{i=1}^{N} C_i^L$, then,

$$\begin{aligned}
Z_{n',m'}(x) &= \sum_{\substack{h \in H \\ h \geq x}} P_{m'}(h) L_{n'}(h) \\
&= \sum_{\substack{h \in H \\ h \geq x}} P_{m'}(h) \prod_{i=1}^{n'} \Delta_i^L(\lfloor h \rfloor_{C_i^L}) \\
&= \sum_{\substack{h \in H \\ h \geq x}} P_{m'}(h) \prod_{i=1}^{n'} \Delta_i^L(\lfloor x \rfloor_{C_i^L}) && \text{by lemma 2.1} \\
&= P_{m'}^{br}(x) \prod_{i=1}^{n'} \Delta_i^L(\lfloor x \rfloor_{C_i^L}) && \text{by definition 2.15}
\end{aligned}$$

which is accomplished by line 4.

No Reweighting Case: If x satisfies, $x \geq \bigvee_{i=x.n+1}^{N} C_i^L$ and $x \geq \bigvee_{i=x.m+1}^{M} C_i^P$, then,

$$\begin{aligned}
Z_{n',m'}(x) &= \sum_{\substack{h \in H \\ h \geq x}} P_{m'}(h) L_{n'}(h) \\
&= \sum_{\substack{h \in H \\ h \geq x}} \left(L_n(h) \prod_{i=x.n+1}^{n'} \Delta_i^L(\lfloor h \rfloor_{C_i^L}) \right) \left(P_m(h) \prod_{i=x.m+1}^{m'} \Delta_i^P(\lfloor h \rfloor_{C_i^P}) \right) \\
&= \sum_{\substack{h \in H \\ h \geq x}} L_n(h) P_m(h) \prod_{i=x.n+1}^{n'} \Delta_i^L(\lfloor h \rfloor_{C_i^L}) \prod_{i=x.m+1}^{m'} \Delta_i^P(\lfloor h \rfloor_{C_i^P}) \\
&= Z_{n,m}(x) \prod_{i=x.n+1}^{n'} \Delta_i^L(\lfloor x \rfloor_{C_i^L}) \prod_{i=x.m+1}^{m'} \Delta_i^P(\lfloor x \rfloor_{C_i^P})
\end{aligned}$$

which is accomplished by line 6. □

Lemma 2.4 (Frontier Calculation)**.** *If* $F = \bigvee_{i=1}^{N'} C_i^L \wedge \left(\bigvee_{i=N+1}^{N'} C_i^L \vee \bigvee_{i=M+1}^{M'} C_i^P \right)$*, then* $x \geq F$ *and* x *is valid at time* (N, M) *imply* x *is valid at time* (N', M')*.*

Proof. If $x \geq F$, then either, $x \geq \bigvee_{i=1}^{N'} C_i^L$ or $x \geq \bigvee_{i=N+1}^{N'} C_i^L \vee \bigvee_{i=M+1}^{M'} C_i^P$. The first case is exactly the same as the "Constant Likelihood" branch in the definition of validity at time (N', M'). So we only need to consider the second case where $x \geq \bigvee_{i=N+1}^{N'} C_i^L \vee \bigvee_{i=M+1}^{M'} C_i^P$.

The other assumption, that x is valid at time (N, M), also has two cases: either, $x \geq \bigvee_{i=1}^{N} C_i^L$ or $x \geq \bigvee_{i=x.n+1}^{N} C_i^L \vee \bigvee_{i=x.m+1}^{M} C_i^P$.

Case 1: If $x \geq \bigvee_{i=1}^{N} C_i^L$ and $x \geq \bigvee_{i=N+1}^{N'} C_i^L \vee \bigvee_{i=M+1}^{M'} C_i^P$, then we may combine the two joins over the likelihood frontiers to write, $x \geq \bigvee_{i=1}^{N'} C_i^L$, which is the "Constant Likelihood" branch for validity at time (N', M').

Case 2: If $x \geq \bigvee_{i=x.n+1}^{N} C_i^L \vee \bigvee_{i=x.m+1}^{M} C_i^P$ and $x \geq \bigvee_{i=N+1}^{N'} C_i^L \vee \bigvee_{i=M+1}^{M'} C_i^P$, then we may combine the joins over the likelihood and prior frontiers to write, $x \geq \bigvee_{i=x.n+1}^{N'} C_i^L \vee \bigvee_{i=x.m+1}^{M'} C_i^P$, which is the "No Reweighting" branch for validity at time (N', M'). □

Theorem 2.3 (Advance Trie Time Validity)**.** *We must show that after running* `AdvanceTrieTime`*, the trie is valid at its new time,* (N', M')*.*

Proof. By lemma 2.4, if $x \geq F$, then x is valid at the new time. Otherwise, x is strictly within F. All such nodes are updated to have the new time $(x.n, x.m) = (N', M')$. By lemma 2.3, these nodes are valid at their new time. □

Theorem 2.4 (Advance Trie Time Correctness)**.** *The* `AdvanceTrieTime` *algorithm preserves the correctness of the trie.*

Proof. We show this by induction on the trie.

We are concerned with nodes, $x \leq F$, since nodes which are beyond the update boundary, $x > F$, are unaffected by the algorithm.

At the start of the algorithm, all nodes are valid and correct.

Base Case: If $x \in F$, the x is valid and correct and we have already shown that running `AdvanceNodeTime` on x preserves its validity and correctness.

Induction Step: If $x < F$, then, because we process nodes in reverse topological order, by the induction hypothesis, all its immediate children are correct and have time (N', M'). Thus line 6 recalculates the unnormalized posterior of x correctly by lemma 2.2. □

2.2 Likelihood Tracking

The preceding algorithm is correct, but we wish to avoid keeping all the Δ_i^L in memory globally and the cost of calculating truncations, $\lfloor x \rfloor_{C_i^L}$. To do so we introduce a field called "unpushed likelihood" which will be used to track the product of likelihood deltas. Furthermore, we limit ourselves to handling a single likelihood update at a time.

Formally, we define augmented nodes with an $x.l$ field, describining the unpushed likelihood. We define an augmented trie corresponding to these nodes.

Algorithm 3 Single Likelihood Update

Require: Bayesian Trie B
Require: B is valid at time (N, M).
1: **for** each node $x < C_{N+1}^L$ in topological order **do**
2: $\quad y.l \leftarrow y.l * x.l \quad \forall y \in x.\text{children}$
3: $\quad x.l \leftarrow 1$
$\quad \triangleright$ For each $x \in C_{N+1}^L$, the product $\prod_{i=x.n+1}^{N} \Delta_i^L(\lfloor x \rfloor_{C_i^L})$ is the unpushed likelihood, $x.l$.
4: **for** each node $x \in C_{N+1}^L$ **do**
5: $\quad x.l \leftarrow x.l * \Delta_{N+1}^L(x)$
6: Run `AdvanceTrieTime` to the new time of $(N+1, M)$.
7: **for** each node $x \in C_{N+1}^L$ **do**
8: $\quad y.l \leftarrow y.l * x.l \quad \forall y \in x.\text{children}$
9: $\quad x.l \leftarrow 1$
10: $B.N \leftarrow B.N + 1$

We define our invariant, "push correctness":

Definition 2.29 (Push Correctness)**.** *We say that a Bayesian trie B is push correct for time (N, M) if, for every node whose descendant likelihoods have changed by a constant factor, that factor is equal to the product of the "unpushed likelihood" fields of all of its ancestors (including itself). I.e.,*

$$\forall x,\, x \geq \bigvee_{i=x.n+1}^{N} C_i^L \implies \prod_{y \leq x} y.l = \prod_{i=x.n+1}^{N} \Delta_i^L\big(\lfloor x \rfloor_{C_i^L}\big)$$

Theorem 2.5 (Single Likelihood Update Push Correctness)**.** *The `SingleLikelihoodUpdate` algorithm preserves push correctness.*

Proof. There are two cases to consider: $x \leq C_{N+1}^L$ and $x > C_{N+1}^L$. For $x \leq C_{N+1}^L$, push correctness is trivially satisfied. By the end of the algorithm, such a node's local time has been updated to $x.n = N+1$ (during the call to `AdvanceTrieTime`), making the empty product on the right-hand side equal to 1. Correspondingly, its unpushed likelihood $x.l$ has been set to 1, leaving the left-hand side product equal to 1.

If $x > C_{N+1}^L$, we rely on the inductive assumption that the trie was push correct at the start of the algorithm. In this case, lines 2 and 3 do not affect the push correctness for these nodes since unpushed likelihood is simply moved from one ancestor to another, leaving the total product $\prod_{y \leq x} y.l$ unchanged. On line 5, we transition from being correct for time (N, M) to being correct for time $(N+1, M)$. For every node $x > C_{N+1}^L$, we correctly multiply the factor $\Delta_{N+1}^L(\lfloor x \rfloor_{C_{N+1}^L})$ into the unpushed likelihood of the ancestor node in C_{N+1}^L. Finally, on lines 8 and 9, we once again merely redistribute the unpushed likelihood among ancestors without changing the total product. □

Lemma 2.5 (Equivalence of Unpushed Likelihood)**.** *Immediately before the call to `AdvanceTrieTime` in `SingleLikelihoodUpdate`, for every node $x \in C_{N+1}^L$, the value $x.l$ is exactly equal to the historical product of likelihood deltas:*

$$x.l = \prod_{i=x.n+1}^{N+1} \Delta_i^L\big(\lfloor x \rfloor_{C_i^L}\big)$$

Proof. Before line 5, the first loop has propagated all unpushed likelihoods down to the nodes in C^L_{N+1}. By the push correctness invariant for time N (which applies since $x \in C^L_{N+1} \implies x \geq \bigvee_{i=x.n+1}^{N} C^L_i$), and because all proper ancestors $y < x$ have had their $y.l$ set to 1, the value of $x.l$ is exactly $\prod_{i=x.n+1}^{N} \Delta^L_i(\lfloor x \rfloor_{C^L_i})$. Line 5 then multiplies $x.l$ by $\Delta^L_{N+1}(\lfloor x \rfloor_{C^L_{N+1}})$, extending the product to $N+1$. □

Corollary 2.1 (Correctness of Efficient AdvanceNodeTime)**.** *In an efficient implementation of* `AdvanceNodeTime` *for a likelihood update, the calculation of the new posterior $x.Z$ for nodes $x \in C^L_{N+1}$ (line 2 of Algorithm 1) can replace the explicit product over historical deltas with a single multiplication by $x.l$:*

$$x.Z \leftarrow x.Z \cdot x.l$$

Because this substitution yields the exact same mathematical value for $x.Z$ by Lemma 2.5, this efficient $O(1)$ implementation trivially inherits the Validity and Correctness theorems proven for the abstract `AdvanceTrieTime`*.*

2.3 Character Priors from Token Models

Note: It is helpful to make three kinds of distinctions: (1) between sets and their elements, (2) between sequences and their elements, and (3) between characters and tokens. We will choose to use uppercase letters for sets, Greek letters for individual characters, and bold font for token sequences. For example, $\mathbf{A}$ would represent a set of token sequences while α would represent a character.

Since Dotter uses a letter-by-letter trie, but language models return token probabilities, we face the task of converting a token model to a literal model.

A given literal prefix corresponds to several token paths. For example, the prefix `sci` could be part of the words with the following tokenizations (using the GPT-2 tokenizer):

- `scientific` (single token),
- `science` (single token),
- `sci-ences` (two tokens),
- `sc-iss-or` (three tokens),

We can see that a literal prefix can correspond to multiple token prefixes (empty string, `sci`, and `sc`), and that for a given token prefix, the literal prefix can be generated from multiple terminal tokens (`science`, `scientific`).

Here we introduce a formalization of tokenization and a key property of byte-pair encoders.

Definition 2.30 (Token)**.** *A token is a non-empty string.*

Definition 2.31 (Token Set)**.** *A token set is a set of tokens. We will use the notation $\mathbf{\Lambda}$ to represent our token set.*

Definition 2.32 (Stop Token)**.** *We will require that our token set contain a token corresponding to the "stop" character, '■'.*

Definition 2.33 (Properly Terminated Token Sequence)**.** *A token sequence is properly terminated if and only if it ends with the stop token, with all preceding tokens not being the stop token. I.e.,*

$$\forall i \quad (i = |\mathbf{a}| - 1 \iff \mathbf{a}[i] = \text{'■'})$$

Analogously to the string case, we will use the notation $\mathbf{H}$ to represent the set of all properly terminated token sequences.

Definition 2.34 (Token Sequence)**.** *A token sequence is a sequence of tokens.*

Definition 2.35 (Literalization Function)**.** *The literalization function, T^{-1}, maps a token sequence to a string, by concatenating its tokens. I.e., $T^{-1}(\mathbf{a}) = Concat_{i<|\mathbf{a}|}(\mathbf{a}[i])$.*

Definition 2.36 (Tokenization Function)**.** *A tokenization function is a mapping from strings to token sequences,* $T : \mathcal{A}^* \to \mathbf{\Lambda}^*$*, that is inverted by literalization, i.e.,*

$$T^{-1}(T(x)) = x \quad \forall x \in \mathcal{A}^*$$

(but the converse will not usually hold, i.e., different token sequences may correspond to the same string.) We also require that the tokenization function maps properly terminated strings to properly terminated token sequences,

$$T : H \to \mathbf{H}$$

Definition 2.37 (Token to Character Index Function)**.** *The token to character index function is a utility function mapping the index of a token within its token sequence to the index of the first character of that token in the corresponding string.*

$$TokenToCharacterIndex(\mathbf{a}, i) = \sum_{j<i} |\mathbf{a}[j]|$$

Proposition 2.3 (Monotonicity of Token to Character Index Function)**.** *The token to character index function is strictly monotonic.*

Proof. This follows from the requirement that tokens are non-empty. □

Definition 2.38 (Character to Token Index Function)**.** *The character to token index function,* $CharToTokenIndex_T(x, j)$*, is a utility function derived from the tokenization function,* T*, mapping the index of a character in the string to the index of the token that contains that character. I.e. it is the greatest value of* i *that satisfies,*

$$TokenToCharacterIndex(T(x), i) \leq j$$

Definition 2.39 (Break Point)**.** *We refer to an index in a string as a "break point" if it is the first character of a token with respect to our tokenization function. So* i *is a break point for a string* x *and a tokenization function* T *if there exists some* j *such that* $i = TokenToCharacterIndex(T(x), j)$*.*

Definition 2.40 (Boundary Sufficiency)**.** *The boundary sufficiency property of a tokenization function states that if* i_1, i_2 *are break points in a string* x*, then the tokenization within the interval* $[i_1, i_2)$ *does not depend on characters outside this interval. Formally, a tokenization function* T *has boundary sufficiency if for all strings* x *and break points* $i_1 \leq i_2$ *of* x*, letting* $j_1 = CharToTokenIndex_T(x, i_1)$ *and* $j_2 = CharToTokenIndex_T(x, i_2)$*, we have*

$$T(x[i_1 : i_2]) = T(x)[j_1 : j_2]$$

Henceforth, we will always assume that our tokenization function has boundary sufficiency. This is because all byte-pair encoders have this property.

We can now formalize a token-based language model.

Definition 2.41 (Conditional Token Prior Function)**.** *Our conditional token prior function tells us the conditional probability of a token given a token sequence. It is a mapping*

$$\mathbf{P} : \mathbf{\Lambda}^* \times \mathbf{\Lambda} \to \mathbb{R}^+$$

written as $\mathbf{P}(\mathbf{t} \mid \mathbf{a})$*, where,*

$$\forall \mathbf{a} \in \mathbf{\Lambda}^*, \sum_{\mathbf{t} \in \mathbf{\Lambda}} \mathbf{P}(\mathbf{t} \mid \mathbf{a}) = 1$$

Definition 2.42 (Conditional Token Prior Function Sequence)**.** *A conditional token prior function sequence is a sequence of conditional token prior functions,* $\{\mathbf{P}_n\}$*.*

Definition 2.43 (Token Branch Prior)**.** *We refer to the prior probability of beginning with a given sequence of tokens as the, "token branch prior", denoted* $\mathbf{P}_n^{br}(\mathbf{a})$*. It is defined as,*

$$\mathbf{P}_n^{br}(\mathbf{a}) = \prod_{i<|\mathbf{a}|} \mathbf{P}_n(\mathbf{a}[i] \mid \mathbf{a}[:i])$$

Once again, we pedantically require calling this a "branch prior", to remind ourselves that priors stricly only apply to properly terminated strings. Note however, that in the case that $\mathbf{a}$ *is properly terminated this is the true prior probability of the string (see below).*

Henceforth we will study prior function sequences that are derived from token prior functions by the relation,

$$P_n(h) = \mathbf{P}_n^{br}(T(h))$$

We can now state our first result

Theorem 2.6 (Token Prior Path Summation)**.** *The branch prior of a string* a*, is the sum of the token branch priors of all token sequences that pass through* a*.*

$$P^{br}(a) = \begin{cases} 1, & a = \epsilon, \\ \sum_{\mathbf{b}\in\mathbf{\Lambda}^*,\, T^{-1}(\mathbf{b})<a} \mathbf{P}^{br}(\mathbf{b}) \sum_{\mathbf{t}\in\mathbf{\Lambda},\, a\le T^{-1}(\mathbf{b}+\mathbf{t})} \mathbf{P}(\mathbf{t} \mid \mathbf{b}), & a \neq \epsilon. \end{cases}$$

Proof. We prove the two cases separately.

If $a = \epsilon$, then by Definition 2.15,

$$P^{br}(\epsilon) = \sum_{\substack{h\in H \\ h\ge\epsilon}} P(h) = \sum_{h\in H} P(h) = 1.$$

Now let $a \neq \epsilon$. Again from Definition 2.15,

$$P^{br}(a) = \sum_{\substack{h\in H \\ h\ge a}} P(h).$$

For each $h \geq a$, write $\mathbf{c} := T(h)$. Since $T^{-1}(\mathbf{c}) = h \geq a$, there exists a unique smallest j such that $a \leq T^{-1}(\mathbf{c}[:j+1])$. Define $\mathbf{b} := \mathbf{c}[:j]$ and $\mathbf{t} := \mathbf{c}[j]$. By minimality of j, we have

$$T^{-1}(\mathbf{b}) < a \leq T^{-1}(\mathbf{b}+\mathbf{t}).$$

So each $h \geq a$ determines a unique pair $(\mathbf{b}, \mathbf{t})$ with those inequalities.

Grouping the sum over h by these pairs gives

$$P^{br}(a) = \sum_{\substack{\mathbf{b}\in\mathbf{\Lambda}^* \\ T^{-1}(\mathbf{b})<a}} \sum_{\substack{\mathbf{t}\in\mathbf{\Lambda} \\ a\le T^{-1}(\mathbf{b}+\mathbf{t})}} \sum_{\substack{h\in H: \\ T(h) \text{ begins with } \mathbf{b}+\mathbf{t}}} P(h).$$

The innermost sum is exactly the prior probability of beginning with $\mathbf{b}+\mathbf{t}$, namely $\mathbf{P}^{br}(\mathbf{b}+\mathbf{t})$. By the token branch prior definition,

$$\mathbf{P}^{br}(\mathbf{b}+\mathbf{t}) = \mathbf{P}^{br}(\mathbf{b})\,\mathbf{P}(\mathbf{t} \mid \mathbf{b}).$$

Substituting yields

$$P^{br}(a) = \sum_{\mathbf{b}\in\mathbf{\Lambda}^*,\, T^{-1}(\mathbf{b})<a} \mathbf{P}^{br}(\mathbf{b}) \sum_{\mathbf{t}\in\mathbf{\Lambda},\, a\le T^{-1}(\mathbf{b}+\mathbf{t})} \mathbf{P}(\mathbf{t} \mid \mathbf{b}),$$

as claimed. □

2.4 Prior Tracking

We now describe an algorithm to efficiently compute the branch prior. We first note that the product over prior deltas in line 2 of algorithm 1 can be expressed as a quotient of branch priors.

Lemma 2.6 (Prior Delta Quotient)**.** *If* $x \geq \bigvee_{i=x.m+1}^{m'} C_i^P$*, then*

$$\prod_{i=x.m+1}^{m'} \Delta_i^P(\lfloor x \rfloor_{C_i^P}) = P_{m'}^{br}(x)/P_m^{br}(x)$$

Proof. □

Thus if we have access to the branch prior function at valid nodes, then we can efficiently compute the prior-related terms in line 2 and line 4 of algorithm 1. The purpose of the prior tracking algorithm is to compute $\mathbf{P}_{M'}^{br}(x)$ for all valid nodes x at prior time M'. We will assume that the inner sum in theorem 2.6 can be computed in $O(1)$ time.

Algorithm 4 Prior Tracking

Require: x ▷ the node to update
Require: M ▷ the prior time for which to compute the branch prior
Require: F ▷ the complete prefix code on which to evaluate the branch priors
1: ϵ.tbp $\leftarrow 1$
2: **for** each $x < F$ in topological order **do**
3: $\quad x$.tbp $\leftarrow x$.token_ancestor.tbp $* \mathbf{P}_{M'}(x.\text{final\_token} \mid T^{-1}(x.\text{token\_ancestor}.value))$
4: **for** each $x \in F$ **do**
5: $\quad x.P \leftarrow \sum_{\mathbf{b} \in \mathbf{\Lambda}^*,\, T^{-1}(\mathbf{b}) < x} \mathbf{P}^{br}(\mathbf{b}) \sum_{\mathbf{t} \in \mathbf{\Lambda},\, x \leq T^{-1}(\mathbf{b}+\mathbf{t})} \mathbf{P}(\mathbf{t} \mid \mathbf{b})$,

When we update a node's prior time as we do in algorithm 1, we should snapshot the branch prior for that node in $x.P_{old}$, which represents $P_{x.m}^{br}(x)$. Then, before the next time we advance the trie time, we should run algorithm 4 first to populate $x.P$, which represents $P_{M'}^{br}(x)$. Their quotient tells us how the prior has changed.

We have two tricks to efficiently compute the sum in line 5 of algorithm 4:

1. We recognize that the eligible longest token prefixes correspond to the prefixes of the last word, since tokens never contain spaces in the middle of them. Additionally we can limit the values of $\mathbf{B}$ to at most 15 ancestor prefixes, since tokens are at most 15 characters long in our tokenizer.
2. We can preconvert the language model's output of logits to an array of *cumulative* probabilities. This allows us to efficiently compute the alphabetical sum over compatible $\mathbf{t}$, by taking the difference between the cumulative probabilities of the start and end of the alphabetical range of $\mathbf{t}$ defined by $x \leq T^{-1}(\mathbf{b} + \mathbf{t})$.

2.5 Maximum Descendant Likelihood

Finally, we discuss how to track the maximum descendant likelihood for nodes. The quantity is not needed to compute posteriors and is orthogonal to the preceding discussion. However, it will be helpful later for determining which token sequences should be evaluated by the language model.

2.6 Global Variables

- `track_mdlfta`: **bool** — indicates whether we are tracking maximum descendant likelihoods for token ancestors. This is only necessary on the server where we will want to be able to evaluate token sequence posteriors.
- `max_token_length`: **int** — the maximum length of a token.

- `pta_lut`: **dict** — a lookup table for possible token ancestors, mapping from (ancestor final token, token prefix) to a boolean indicating whether it is possible for any token beginning with the token prefix to follow upon the ancestor final token.
- `llm_results`: **dict** — a mapping from (token sequence, token) to the language model's conditional probability of the token given the token sequence.
- `llm_fans`: **dict** — a mapping from (token sequence, token prefix) to the language model's conditional probability that the next token begins with the token prefix given the token sequence.

2.7 Fields of a Node

A node in the trie has the following fields. (Note: All array fields have length `max_token_length` (0-indexed) unless otherwise specified.)

Constant Fields:

- `letter`: **str** — the character represented by this node.
- `value`: **str** — the string represented by this node, i.e., the concatenation of the parent's value and the character represented by this node. (In implementation, we only need at most the last 2*`max_token_length` characters of the value.)
- `depth`: **int** — the depth of this node in the trie.
- `final_token`: **str** — the token to which this node belongs, if this node is the last character of a token.
- `final_token_length`: **int** — the number of characters in the final token.
- `ancestor_final_tokens`: **list of strs** — the i-th ancestor final token is the final token of the i-th ancestor. (In implementation, this can be an array of the ancestor final token lengths, and we may slice into our value to derive the final token.)
- `possible_token_ancestors`: **list of bool** — a list of flags indicating whether the i-th ancestor could possibly be a token ancestor of this node. (See chapter 8.)

Mutable Fields:

- `posterior_Z`: **float** — the unnormalized posterior probability of this node being a prefix of the user's target string,
- `ever_parent_of_visible_node`: **bool** — indicates whether this node has ever been a parent of a visible node (at a time when a signal was received). This must be false for the likelihood field to be valid. If this flag is true, the likelihood can only be calculated as a weighted sum of its children's likelihoods, $P(D|S \leq X) = \sum_l P(D|S + l \leq X)P(S + l \leq X|S \leq X)$, since its children have different likelihoods. Hence, if a node was not ever a parent of a visible node, then its likelihood is known absolutely, but if a node was once a parent of a visible node, then its likelihood is at best known conditional on a prior model. Since our prior model mutates over time, this is not definite knowledge of the likelihood.
- `prior`: **float** — the prior probability of this node being a prefix of the user's target string.
- `break_prior`: **float** — the prior probability that not only is this node a prefix of the user's target string, but also that it is the last character of a token in the tokenization of the user's target string.
- `ancestor_break_priors`: **list of floats** — the i-th ancestor break prior is the `break_prior` of the i-th ancestor. This may be stale.
- `unpushed_break_priors`: **list of optional floats** — the i-th element is the fresh `break_prior` if the i-th ancestor's `break_prior` has changed since the last time we visited this node.
- `token_fans`: **list of floats** — the i-th token fan is the prior probability of this node being a prefix of the target string conditional on its i-th parent being the last character of the token immediately preceding the token to which this character belongs.
- `final_token_prior`: **float** — the conditional prior probability of the final token if the ancestor at depth `depth - final_token_length` is the last character of a token.
- `unpushed_llm_results`: **list of optional bool** — the i-th element is true if the `llm_result` corresponding to the value of the i-th ancestor has changed since the last time we visited this node.

- `likelihood`: **optional float** — the accumulated likelihood of all signals conditional on this node being a prefix of the user's target string, $\prod_{d \in D} P(d|S \leq X)$.
- `unpushed_likelihood`: **float** — likelihood that has not been accumulated to descendant nodes.
- `max_descendant_likelihood_for_token_ancestor`: **list of floats** — the i-th element is the maximum (valid) likelihood of any descendant of this node which could also have this node's i-th ancestor as a token ancestor. (See chapter 8.)

2.8 Valid Descent

The above data structure gives us the machinery to effectively defer the calculation of the posteriors of most nodes.

When we visit a node x, we update the statistics of its children. If the node x has no children because it has never been visited before, then we initialize its children.

Algorithm 5 Initialize Children

Require: Node x

x.unpushed_likelihood $\leftarrow 1$
x.unpushed_break_priors$[i] \leftarrow$ None $\quad \forall i$
x.unpushed_llm_results$[i] \leftarrow$ None $\quad \forall i$
for each character c in the alphabet **do**
    Create child node y of x
    y.letter $\leftarrow c$
    y.value $\leftarrow x$.value $+ c$
    y.depth $\leftarrow x$.depth $+ 1$
    Tokenize the string corresponding to y into a sequence of tokens $\mathbf{T} = [\mathbf{t}_1, \ldots, \mathbf{t}_k]$
    y.final_token $\leftarrow \mathbf{t}_k$
    y.final_token_length $\leftarrow |\mathbf{t}_k|$
    y.ancestor_final_tokens$[0] \leftarrow y$.final_token
    y.ancestor_final_tokens$[i] \leftarrow x$.ancestor_final_tokens$[i-1] \quad \forall i \geq 1$
    y.ever_parent_of_visible_node $\leftarrow$ false
    y.ancestor_break_priors$[i] \leftarrow x$.ancestor_break_priors$[i-1] \quad \forall i \geq 1$
    y.unpushed_break_priors$[i] \leftarrow$ None $\quad \forall i$
    y.unpushed_llm_results$[i] \leftarrow$ None $\quad \forall i$
    y.final_token_prior $\leftarrow$ `llm_results`$[(y.\text{value}[: -y.\text{final\_token\_length}],\ y.\text{final\_token})]$
    y.break_prior $\leftarrow y$.ancestor_break_priors$[y$.final_token_length$]$
        $* y$.final_token_prior
    y.ancestor_break_priors$[0] \leftarrow y$.break_prior
    y.token_fans$[i] \leftarrow$ `llm_fans`$[(y.\text{value}[: -i],\ y.\text{value}[-i :])] \quad \forall i$
    y.prior $\leftarrow \sum_i y$.ancestor_break_priors$[i] * y$.token_fans$[i]$
    y.likelihood $\leftarrow x$.likelihood
    if `track_mdlfta` **then**
        y.possible_token_ancestors$[i] \leftarrow$ `pta_lut`$[y$.ancestor_final_tokens$[i],\ y$.value$[-i :]] \quad \forall i$
            ▷ This correctly sets y.possible_token_ancestors$[0]$ to true.
        y.max_descendant_likelihood_for_token_ancestor$[i] \leftarrow$

$$\begin{cases} x.\text{likelihood} & \text{if } y.\text{possible\_token\_ancestors}[i] \\ 0 & \text{otherwise} \end{cases} \quad \forall i$$

        $TrackMDLFTA(y)$ Using algorithm 7.
    y.unpushed_likelihood $\leftarrow 1$
    y.posterior_Z $\leftarrow y$.prior $* y$.likelihood
x.likelihood $\leftarrow$ None

When we visit a node, we push the unpushed likelihood and the unpushed break priors to its children. Let x be our current node, and y be a child of x.

Algorithm 6 Visit Node

Require: Node x

for each child y of x **do**
  `old_prior` $\leftarrow y$.prior
  if y.ever_parent_of_visible_node is false **then**
    `old_likelihood` $\leftarrow y$.likelihood
    y.likelihood $*= x$.unpushed_likelihood
  y.unpushed_likelihood $*= x$.unpushed_likelihood
  y.unpushed_llm_results$[i] \vee= x$.unpushed_llm_results$[i] \quad \forall i$
  y.unpushed_break_priors$[i] \vee= x$.unpushed_break_priors$[i] \quad \forall i$
  y.token_fans$[i] \leftarrow$ `llm_fans`$[(y.\text{value}[:-i],\ y.\text{value}[-i:])]$
    $\forall i$ s.t. x.unpushed_llm_results$[i] \neq$ None
  `prior_changed` $\leftarrow (\exists\ i:\ x.\text{unpushed\_break\_priors}[i] \neq \text{None}) \vee (\exists\ i:\ x.\text{unpushed\_llm\_results}[i] \neq \text{None})$
  if `prior_changed` **then**
    y.prior $\leftarrow \sum_i y$.ancestor_break_priors$[i] * y$.token_fans$[i]$
  if x.unpushed_llm_results[x.final_token_length] is not None **then**
    y.final_token_prior $\leftarrow$ LLM_Results[y.value[: $-y$.final_token_length]][y.final_token]
    y.break_prior $\leftarrow y$.ancestor_break_priors[y.final_token_length] $* y$.final_token_prior
    y.ancestor_break_priors[0] $\leftarrow y$.break_prior
    y.unpushed_break_priors[0] $\leftarrow y$.break_prior
  if y.ever_parent_of_visible_node is false **then**
    y.posterior_Z $\leftarrow y$.prior $* y$.likelihood
  else
    `prior_correction` $\leftarrow$ `y.prior/old_prior`
    `likelihood_correction` $\leftarrow x$.unpushed_likelihood
    y.posterior_Z $*=$ `prior_correction` $*$ `likelihood_correction`
  if `track_mdlfta` is true and y.ever_parent_of_visible_node is false **then**
    y.max_descendant_likelihood_for_token_ancestor$[i] \leftarrow$

$$\begin{cases} y.\text{likelihood} & \text{if } y.\text{possible\_token\_ancestors}[i] \\ 0 & \text{otherwise} \end{cases} \quad \forall i$$

    $TrackMDLFTA(y)$ Using algorithm 7.
x.unpushed_likelihood $\leftarrow 1$
x.unpushed_break_priors$[i] \leftarrow$ None $\quad \forall i$
x.unpushed_llm_results$[i] \leftarrow$ None $\quad \forall i$

Algorithm 7 Track Token Descendant Max Likelihood

Require: Node x
Require: x.max_descendant_likelihood_for_token_ancestor[i] is already correct for all i

```
for each i < max_token_length do
    new_mdlfta ← x.max_descendant_likelihood_for_token_ancestor[i]
    node ← x
    dist_to_last_token ← i
    while true do
        old_mdlfta ← node.max_descendant_likelihood_for_token_ancestor[dist_to_last_token]
        if old_mdlfta < new_mdlfta then
            node.max_descendant_likelihood_for_token_ancestor[dist_to_last_token] ← new_mdlfta
        else
            break
        if dist_to_last_token = 0 then
            dist_to_last_token ← node.final_token_length
        dist_to_last_token − = 1
        node ← node.parent
        if node is None then
            break
```

Algorithm 8 Valid Descent

Require: Bayesian Trie B and node x in B.

```
if x is a leaf in B then
    InitializeChildren(x) Using algorithm 5.
else
    VisitNode(x) Using algorithm 6.
```

Algorithm 9 Ensure Rooted Subgraph

Require: Bayesian Trie B, and a trie V which is a subgraph of B sharing the root.

```
Let Q be a queue initialized with the root of V.
while Q is not empty do
    Pop v from Q
    Let b be the node in B corresponding to v
    ValidDescent(B, b) Using algorithm 8.
    for each child u of v in V do
        Push u to Q
```

Algorithm 10 Up Propagation

Require: Bayesian Trie B and rooted subgraph V of B.

```
Sort the nodes in V in reverse topological order.
for each node v in the reverse topological order do
    if v is not a leaf in V then
        Set v.post_Z ← ∑_{u∈v.children} u.post_Z
```

Definition 2.44. *We define the truncation of a token sequence, $\lfloor \mathbf{B} \rfloor_d$, to be the longest token sequence prefix of* $\mathbf{B}$*, which has fewer than d characters, i.e., it is the longest* $\mathbf{C}$ *such that $|Literalize(\mathbf{C})| < d$.*

Definition 2.45. *For a given character sequence, S, and a given token sequence,* $\mathbf{B}$*, (which are compatible with each other in the sense that $\mathbf{B} \leq S$), we define the "Chain Maximum Likelihood" with respect to depth*

d, to be the maximum likelihood of any descendant which is compatible with both the character sequence and the token sequence up to depth d.

$$ChainMaxLikelihood(S, \mathbf{B}, d) = \max_{\substack{Q \geq S[:d] \\ Tokenize(Q) \geq \lfloor \mathbf{B} \rfloor_d}} P(D \mid Q)$$

Theorem 2.7. *For a given character sequence, S, and a given token sequence, $\mathbf{B}$, the Chain Maximum Likelihood is a decreasing function of the depth.*

Proof. The two conditions become more restrictive, i.e.,

$$\begin{aligned} Q \geq S[: d+1] &\implies Q \geq S[: d] \\ Tokenize(Q) \geq \lfloor \mathbf{B} \rfloor_{d+1} &\implies Tokenize(Q) \geq \lfloor \mathbf{B} \rfloor_d \end{aligned}$$

□

Theorem 2.7 is necessary to prove that breaking at line 10 in algorithm 7 is correct.

2.9 Lazy Updates

The purpose of an update is to assimilate a change to the likelihood or prior so that the trie remains in a valid state, i.e., the trie will continue to yield correct posterior probabilities if explored via the "valid descent" algorithm.

An important necessary condition for an update to be valid is that it correctly updates the unnormalized posterior probability of the root node.

Each update must be performed in four phases:

1. Select intial frontier: we choose a set of nodes from which to begin the algorithm.
2. Down Propagation: we descend the trie until we reach a "valid update leafset" of nodes, i.e., a set of nodes which is a complete prefix partition and for which we can know the unnormalized posteriors of each node. Efficiency demands that we recognize once we have reached a valid leafset and stop descending as early as possible.
3. Fix Leafset: we fix the unnormalized posteriors of all the nodes in the valid update leafset.
4. Up Propagation: we ascend the trie, accumulating unnormalized posteriors

The trick is to choose the smallest prefix partition for which we can truly calculate the unnormalized posteriors. The unnormalized posterior is a sum over one's children's unnormalized posteriors:

How do we know when we have reached a valid update leafset? We assume that after receiving an update, `posterior_Z`, that is, $P_0(S|D)$, has been invalidated for every node in the trie. We hope to derive $P_1(S|D)$, without summing over the children's posteriors,

$$P_1(S|D) = \sum_i P_1(S_i|D).$$

In general if one term is reweighted, we will have to recalculate, but there are four cases in which we can avoid recalculation and descent:

1. The likelihood (and prior) of S is known explicitly,

$$P_1(S|D) = P_1(S)P_1(D|S) \tag{2.1}$$

2. Neither term (neither likelihood nor prior) changes.

$$P_1(S|D) = P_0(S|D) \tag{2.2}$$

3. The likelihoods change by a constant factor l, and the priors do not change.

$$\begin{aligned} P_1(S|D) &= \sum_i P_1(S_i|D) \\ &= \sum_i P_1(S_i) P_1(D|S_i) \\ &= \sum_i P_1(S_i) \big(l \cdot P_0(D|S_i)\big) \\ &= l \sum_i P_0(S_i) P_0(D|S_i) \\ &= l \cdot P_0(S|D) \end{aligned} \tag{2.3}$$

4. The priors change by a constant factor p, and the likelihoods do not change,

$$\begin{aligned} P_1(S|D) &= \sum_i P_1(S_i|D) \\ &= \sum_i P_1(S_i) P_0(D|S_i) \\ &= \sum_i \big(p \cdot P_0(S_i)\big) P_0(D|S_i) \\ &= p \sum_i P_0(S_i) P_0(D|S_i) \\ &= p \cdot P_0(S|D) \end{aligned} \tag{2.4}$$

In order to find the smallest prefix partition for the Up Propagation step, we consider the set of all nodes that satisfy any of these three cases and then take the minimal elements of this set (resulting in an antichain / frontier of the tree).

2.10 Likelihood Update

After receiving a likelihood result from the user interface, we must update the trie to increase the posterior probabilities of the nodes most likely indicated by the user's signal.

Step 1: Select initial frontier: We choose the set of all nodes that were leaves in the visible trie, along with all invisible nodes that have a visible sibling. Equivalently, we are choosing the minimal elements of the set of nodes where every node has no visible children, i.e., has a valid likelihood for this observation, D_i. (We should descend from the root to gather these nodes to ensure that unpushed priors have been updated correctly, i.e., to ensure that every node in this set has a correctly calculated prior.)

Step 2: Down Propagation: Our initially selected frontier is actually a valid update leafset, so no down propagation is necessary. Our initially selected frontier is a valid leafset because it satisfies case 3 above: the likelihood of a given node in this set may not be known explicitly (since it may have been the parent to visible children at some time), but we know how its implicit (true) likelihood has changed and we known that its prior has not changed.

Step 3: Fix Leafset: We fix the unnormalized posteriors of all the nodes in the valid update leafset. We guarantee a valid unnormalized posterior by setting the `post_Z` of each node to,

$$x.\text{post\_Z} = x.\text{post\_Z} * \text{new\_likelihoods[x.value]}$$

Step 4: Up Propagation: We ascend the trie, accumulating unnormalized posteriors of the parents as

$$x.\text{post\_Z} = \sum_{y \in x.\text{children}} y.\text{post\_Z}$$

Algorithm 11 Complete Visible Trie

Require: Trie V of visible nodes

for each node $v \in V$ **do**
  if v is a leaf in V **then**
    continue ▷ Skip if it is already a leaf
  for each character c in the alphabet **do** ▷ Propagate likelihood to invisible children
    if $(v + c) \notin V$ **then**
      Add $(v + c)$ to V
      Set $(v + c)$.new_likelihood $\leftarrow v$.new_likelihood
return V

Algorithm 12 Likelihood Update

Require: Bayesian Trie B, trie of visible nodes, V, with likelihoods for latest observation.

$V \leftarrow$ CompleteVisibleTrie(V) Using algorithm 11.
EnsureRootedSubgraph(B, V) Using algorithm 9.
for each node $v \in V$ **do**
  Set b to be the node in the bayesian trie corresponding to v
  if v is a leaf in V **then**
    b.likelihood $*= v$.new_likelihood
    b.post_Z $*= v$.new_likelihood
    if `track_mdlfta` **then**
      b.max_descendant_likelihood_for_token_ancestor$[i] *= v$.new_likelihood $\quad \forall i$
      $TrackMDLFTA(b)$ Using algorithm 7.
UpPropagation(B, V) Using algorithm 10.

Maintaining maximum descendant likelihoods

For every node in our update leafset we factor in the new likelihood not only to the likelihood of the given node, but also to all elements of the node's array `max_descendant_likelihood_for_token_ancestor` as well as the scalar, `max_token_descendant_likelihood`. when we visit a node during up propagation we immediately accumulate these quantities in the node's possible ancestors.

Possible Token Ancestors

A possible token ancestor of a character sequence, S, is a token sequence, $\mathbf{A}$, such that there is some string Y, such that S is a prefix of Y, $S \leq Y$, and $\mathbf{A}$ is a token sequence prefix of the full tokenization of Y, and is the longest such prefix for which $\mathbf{A} \leq S$. In other words, it is possible that the last token of $\mathbf{A}$ is the last complete token in the tokenization of Y before the last character of S.

A necessary condition is that $(S - \mathbf{A})$ is a character sequence prefix of some token $\mathbf{t}$. Designating the last token of $\mathbf{A}$ as $\mathbf{s} = \mathbf{A}[-1]$, the necessary and sufficient condition is that there exists some token $\mathbf{t}$ such that $(S - \mathbf{A})$ is a prefix of $\mathbf{t}$, and tokenizing the string formed by concatenating $\mathbf{s}$ and $\mathbf{t}$ yields exactly the two tokens $\mathbf{s}$ and $\mathbf{t}$ in order, i.e., $\texttt{Tokenize}(\mathbf{s} + \mathbf{t}) = [\mathbf{s}, \mathbf{t}]$.

Evaluating this predicate for given $\mathbf{s}$ and $(S - \mathbf{A})$ has resisted my attempts to find an efficient algorithm. However, it can be solved in $O(1)$ time by precomputing a lookup table for all possible pairs of tokens, $\mathbf{s}$, and token prefixes, $(S - \mathbf{A})$. This table has size $O(V^2)$, where V is the vocabulary size. If the vocabulary size is 50,000 and we require one bit of memory per entry, this amounts to only about 300MB.

Interestingly for a random choice of $\mathbf{s}$ and $\mathbf{t}$, the probability that $\mathbf{s} + \mathbf{t}$ tokenizes to $[\mathbf{s}, \mathbf{t}]$ is about 1/2. Even knowing $\mathbf{s}$ does not help very much: the conditional entropy is still about 0.6 bits. This result holds when we restrict $\mathbf{t}$ to tokens that are not prefixes of other tokens. Since a substantial fraction of the possible values for $(S - \mathbf{A})$ will be of this form, this indicates that we cannot effectively make a compression of the lookup table.

However, since this is only needed for the step of determining the most likely break nodes, which occurs on the server, we can afford to use a lookup table, since the server has to keep a language model in memory anyway.

2.11 Prior Update

After receiving an "LLM Result" we must update the trie to reflect this refinement of the language model.

Step 1: Select initial frontier: The "LLM Result" corresponds to a token sequence, $\mathbf{A}$. We choose the corresponding character sequence, S, in the bayesian trie as our initial frontier.

Step 2: Down Propagation: We descend from the initial frontier via valid descent until we reach a valid update leafset. A node is known to belong to the valid update leafset, and no further descent from that node is necessary, if and only if the node satisfies one of the following two cases:

- **Valid Likelihood**: The node has never been a parent to a visible node, i.e., it has a valid likelihood.
- **Space Character**: The node is a space character. Formally, the condition is that there exists some token following upon $\mathbf{A}$ which every possible suffix of our node S, must contain, i.e., we require that S satisfy,

$$\exists \mathbf{B}, \mathbf{B} > \mathbf{A}, |\mathbf{B}| = |\mathbf{A}| + 1, \forall Y \geq S, \mathbf{B} < Tokenize(Y)$$

  but in practice we implement this by recognizing that this condition is always true of the space character and rarely true otherwise.

Step 3: Fix Leafset: How we fix the unnormalized posterior of an update leafset node depends on which of the two cases above it satisfies. If it has a valid likelihood, then the likelihood and prior are known explicitly (case 1 above) and we can set the `post_Z` to,

$$S.\text{post\_Z} = S.\text{prior} * S.\text{likelihood}$$

If it is a space character, then we know that this node's prior and the priors of all its descendants must pass through the same token sequence, $\mathbf{B} > \mathbf{A}$. For this node and for all descendants Y, we have,

$$P_1(Y) = P_0(Y) * \frac{P_1(\mathbf{B}|\mathbf{A})}{P_0(\mathbf{B}|\mathbf{A})}$$

since this is the only factor in the product of conditional probabilities that has changed. We can determine the relevant predicted token as, $\texttt{shared\_token} = Tokenize(S)[|\mathbf{A}|]$ and so the correction factor is,

$$\texttt{correction\_factor} = \frac{P_1(\mathbf{B}|\mathbf{A})}{P_0(\mathbf{B}|\mathbf{A})} = \frac{\text{LLM\_Result}[\texttt{shared\_token}]}{\text{Old\_LLM\_Result}[\texttt{shared\_token}]}$$

and we can set the `post_Z` to,

$$S.\text{post\_Z} = S.\text{post\_Z} * \texttt{correction\_factor}$$

Step 4: Up Propagation: We ascend the trie, accumulating unnormalized posteriors of the parents as

$$S.\text{post\_Z} = \sum_{Y \in S.\text{children}} Y.\text{post\_Z}$$

Algorithm 13 Prior Update

Require: Bayesian Trie B, Node S (corresponding to token sequence $\mathbf{A}$), New_LLM_Result, Old_LLM_Result

Let Q be a queue initialized with S
Let L be an empty list ▷ Valid update leafset ▷ **Step 2: Down Propagation**
while Q is not empty **do**
    Pop node x from Q
    ValidDescent(B, x) Using algorithm 8.
    if x.ever_parent_of_visible_node is false $\vee$ x.letter is a space character **then**
        Add x to L
    else
        for each child y of x **do**
            Push y to Q
▷ **Step 3: Fix Leafset**
for each node $x \in L$ **do**
    if x.ever_parent_of_visible_node is false **then**
        x.post_Z $\leftarrow$ x.prior $*$ x.likelihood
    else ▷ x.letter is a space character
        $\texttt{shared\_token} \leftarrow Tokenize(x.\text{value})[|\mathbf{A}|]$
        $\texttt{correction\_factor} \leftarrow \frac{\text{New\_LLM\_Result}[\texttt{shared\_token}]}{\text{Old\_LLM\_Result}[\texttt{shared\_token}]}$
        x.post_Z $*=$ $\texttt{correction\_factor}$
▷ **Step 4: Up Propagation**
Let V be the subgraph of B formed by the paths from the root to every node in L
UpPropagation(B, V) Using algorithm 10.

Note Bene: Why is descending to the space character necessary? After all, if we know exactly how a node's prior has changed, and we know that its likelihood has not changed, then we should be able to correctly recalculate the unnormalized posterior, just as how in the likelihood update we could recalculate the unnormalized posterior of a node if we knew how its likelihood had changed and that its prior had remained constant. The subtle fallacy in this thinking is that the *likelihood of the node has changed*. Although we often think of the likelihood as being independent of the prior, this is only true for pure hypotheses. For a composite hypothesis, like this node we have,

$$P(D|S) = \sum_i P(D|S_i)P(S_i|S)$$

where S_i are the children of S. Since the weighting, i.e., the prior, has changed, so has the *likelihood* of this node. Thus we do not satisfy case 4 above, because the likelihood has changed.

We update the posterior probabilities in two ways:

1. **Likelihood updates**: when the user makes a gesture, the likelihoods of the prefixes are updated. This is a traditional update.
2. **Prior updates**: when the language model server returns new or better prior probabilities, we must recalculate the posteriors.

	Likelihood Update	**Prior Update**
Mathematical Term	$P(D \mid X)$	$P(X)$
Recompute Posteriors	Yes	Yes
Resize Boxes	Yes	Yes
How Many New Nodes?	Dozens	Zero to a few
How to Set Phases	Optimized / gradient descent	Random (or ancestor / minimum placement)
Human-Initiated	Yes	No
Show Immediately	Yes	No — wait for ancestor to finish
Branch of Tree Affected	Entire tree	Single branch

Table 2.1: Comparison of likelihood updates and prior updates.

Chapter 3

Determine Visible Nodes

In this step we identify which prefixes have sufficient posterior probability to be visually displayed.

The two questions are which prefixes to show and whether to show them immediately or to wait until showing them.

3.1 Setting the Visibility Threshold

The visibility threshold will determine the number of visible nodes, which entails a trade-off.

Advantages of a high threshold and few visible nodes:

- There is less clutter and it is less overwhelming to the user.
- Nodes are larger, avoiding the need for small fonts and timers. The user can recognize timers more easily.
- Faster computation and rendering.

Advantages of a low threshold and many visible nodes:

- More granular targets allows us to spread out the probability more evenly, i.e., we can set the timers to make a better approximation to the uniform distribution. (Refer to chapter 5.)

Thus, we want to set the visibility threshold as high as we can while still being able set the timers to approximate a uniform distribution.

3.2 Searching for Visible Nodes

In order to identify all nodes exceeding the visibility threshold, we descend the trie. We know that posterior probability decreases monotonically with depth, hence we may stop descending upon reaching a node whose (normalized) posterior probability is less than the visibility threshold. It is necessary to descend the trie in the manner described in chapter 2, since this guarantees that the unnormalized posterior probabilities we observe are valid. (Searching for visible nodes with a flat scan over all nodes would be invalid, since many of their posteriors would be stale, since their updates have been deferred.)

3.3 Responding to Likelihood Updates versus Prior Updates

Every time our trie is updated, we need to recalculate the visible nodes. Thus the set of visible nodes will change in response to prior updates as well as in response to likelihood updates. However, these should not be handled in the same way. Since a likelihood update is triggered by the user's gesture, the user expects that the visible nodes will change in response. On the other hand, a prior update is triggered by the background evaluation of a language model. Immediately redrawing in response to these updates would be jarring and disorienting. It could be especially problematic if a child appeared right before the user was about to select

its parent, causing the user to make an error (select the parent instead of the child), through no fault of their own. Similarly, it would be problematic if a node disappeared right as the user was about to select it.

To mitigate this issue, we enforce a couple of rules about redrawing the trie in response to prior updates:

- A new visible node cannot be drawn while its parent is close to culmination.
- A visible node can not be removed until the next likelihood update.

Chapter 4

Render Trie

4.1 Embellishments

4.1.1 Branches of the Tree

Chapter 5

Stimulus Optimization: Ideal Timer Placement

5.1 The Continuous One Bit Channel

5.1.1 Two Interpretations

The continuous one bit channel can be conceived in two ways: as a continuous channel with a one bit range, or as a series of times of identical events. The equivalency follows from the fact that the continuous channel may be characterized by the times at which its signal changes. To have a finite (differential) capacity we must constrain the channel to a minimum duration of signal, or equivalently, a minimum time between events. For our purpose, we prefer the latter interpretation of the one bit channel, since the user communicates using a sequence of identical events such as blinks or taps.

5.1.2 The Ideal Timing Distribution

What then, is the capacity of the one bit channel? We prefer the event interpretation for analysis. We consider the ideal distribution of the duration between events. Let T be the minimum duration between events and X be the additional delay beyond the minimum, so that the total duration between events is $T + X$. Let $A = T + E[X]$ be the expected inter-arrival time. The rate is given by

$$C = \max_{\mathcal{P}_X} \frac{h(X)}{A}.$$

We refer to the rate-maximizing distribution of X, $\mathcal{P}_X$, as the ideal timing distribution. We will proceed to show that the ideal timing distribution is an exponential distribution.

We treat the capacity, expected value, and differential entropy as functionals of the probability density function $f(x)$. To find the optimal distribution, we introduce a Lagrange multiplier μ for the normalization constraint $\int f(x)dx = 1$ and define the functional $J(f)$:

$$J(f) = \frac{h(X)}{A} - \mu \left(\int f(x)dx - 1 \right)$$

We maximize $J(f)$ by setting the variational derivative with respect to $f(x)$ to zero:

$$\frac{\delta J}{\delta f(x)} = 0$$

Using the quotient rule for variational derivatives, and noting that $\frac{\delta h(X)}{\delta f(x)} = -1 - \ln f(x)$ and $\frac{\delta A}{\delta f(x)} = \frac{\delta E[X]}{\delta f(x)} = x$, we have:

$$\frac{\delta}{\delta f(x)} \left(\frac{h(X)}{A} \right) = \frac{A(-1 - \ln f(x)) - h(X)x}{A^2}$$

Substituting this into the stationarity condition:

$$\frac{A(-1-\ln f(x))-h(X)x}{A^2}-\mu=0$$

Solving for $f(x)$:

$$f(x)\propto\exp\left(-\frac{h(X)}{A}x\right)$$

proving that X is exponentially distributed with rate parameter equal to the rate of the channel (i.e., $\lambda^*=C$). This fact lets us solve for the rate,

$$C=\frac{1-\ln C}{T+1/C}$$

which by algebra has the solution,

$$C=W(T)/T$$

, where W is the Lambert W function.

A Geometric Intuition

The above result (that the ideal timing distribution is exponential) may be appreciated more readily with a geometric argument. We consider a large sample, $\mathbf{X}$, in N-dimensional space, representing the sequence of additional delays X_i. By the law of large numbers, its L_1 norm, $\|\mathbf{X}\|_1=\sum_i X_i$, must be tightly distributed about the N-simplex, $\|\mathbf{X}\|_1=NE[X]$. This can be thought of as the permissible region for $\mathbf{X}$. Thus the entropy maximizing distribution for $\mathbf{X}$ should assign a uniform distribution over the N-simplex, which is precisely what the exponential distribution does as it depends only on the L_1 norm, $f(\mathbf{X})\propto\exp(-\lambda\|\mathbf{X}\|_1)$.

(Incidentally, this conception also provides a geometric proof of the fact that the differential entropy of the exponential distribution is $1-\ln\lambda$, since the entropy of a uniform distribution over the N-simplex should be the logarithm of its volume. Since its side length is N/λ, its volume is $(N/\lambda)^N/N!$, and its entropy per sample (by Stirling's approximation) is

$$h(X)=\lim_{N\to\infty}\frac{1}{N}\left(N(\log(N-\log\lambda)-N(\log N-1))\right)=1-\ln\lambda$$

as expected.)

5.2 Periodic Signals

5.2.1 Random Scan Times

5.2.2 Periodic signals

5.2.3 Approximating A Uniform Distribution with a Gaussian Mixture

UNFINISHED

We have considered the ideal timing distribution and the periodic timing distribution. How can we induce a uniform distribution on the user's signal? There are several targets that the user may aim for, corresponding to all the prefixes displayed. Each target has an assigned signal time, μ_i and a fixed and known (mixing) probability, π_i. Given the noise $\mathcal{E}\sim\mathcal{N}(0,\sigma)$, the distribution of the observed signal is a mixture of gaussians with means mu_i and constant variance σ^2. Given the mixing probabilities, π_i, how should we position the μ_i to maximize the entropy of the signal?

Intuitively we want to space apart the μ_i to approximate a uniform distribution. Those components with higher pi_i should receive more space.

As soon as we receive a signal we recompute the posterior probabilities of the targets and must assign new phases to them. Hence this computation is on the critical path and ought to be computed in a few

milliseconds. This rules out heavier techniques like gradient descent on a numerical approximation of the entropy.

We discuss two heuristic approaches to the problem as well as an approximation for the entropy which may computed efficiently.

Two Heuristics

UNFINISHED We wish to space the components of the mixture as to approximate a uniform distribution. One heuristic is to determine (by binary search) a value C such that

$$\sum_i 2w_i = P$$

where the widths allotted to each component, w_i, are given by

$$w_i = \mathrm{Re}\left(\sigma\sqrt{2\left(\ln\left(\frac{\pi_i}{\sqrt{2\pi}\sigma}\right) - C\right)}\right)$$

where P is the length of the interval. The means are then given by $\mu_i = w_i + \sum_{j=1}^{i-1} 2w_j$.

This heuristic is derived from approximating the entropy of the mixture as $h(X) \approx -\ln f_{\min}$ where $f_{\min}$ is the minimum value of the mixture density, and approximating the mixture by its greatest component at a given point,

$$f(x) \approx \max_i f_i(x)$$

The value of C represents the minimum log density of the approximated mixture, i.e., the minimum over the interval of the maximum log density of the components. The width $2w_i$ is the range over which the ith component may provide a log density greater than C. If $\ln f_i(\mu_i) < C$, where $f_i(\mu_i)$ denotes the maximum density of the ith component at $x = 0$, then clearly $w_i = 0$, otherwise it is as given above. Our algorithm thus maximizes the minimum density of the maximum component over the interval. Although somewhat crude, this heuristic is efficient.

A refinement may be made by instead approximating the mixture by the sum of its two greatest components, and continuing to try to maximize the minimum density of the distribution. In this case, adjacent components should be spaced apart such that the log of the sum of their densities attains a minimum value of C. Given two adjacent components, i and $i+1$, we are interested in their separation, l_i. Letting $\mu_i = 0$ and $\mu_{i+1} = l_i$, we may solve for the minimum value of the two-component mixture:

$$f(x) = \pi_i \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{x^2}{2\sigma^2}\right) + \pi_{i+1} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-l_i)^2}{2\sigma^2}\right)$$

Note that completing the square for the sum of squared distances gives:

$$(x-\mu_1)^2 + (x-\mu_2)^2 = 2\left(x - \frac{\mu_1+\mu_2}{2}\right)^2 + \frac{(\mu_1-\mu_2)^2}{2}$$

Taking the derivative with respect to x and setting it to zero we have,

$$\frac{d}{dx} f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \left(\frac{-2x\pi_i}{2\sigma^2} \exp\left(-\frac{x^2}{2\sigma^2}\right) - \frac{-2(x-l_i)\pi_{i+1}}{2\sigma^2} \exp\left(-\frac{(x-l_i)^2}{2\sigma^2}\right)\right) = 0$$

Algebra yields,

$$\ln\left(\frac{Rx}{l_i - x}\right) = \frac{x l_i}{\sigma^2} - \frac{l_i^2}{2\sigma^2}$$

, where $R = \pi_i/\pi_{i+1}$. Taken with the condition that $\ln f(x) = C$ the two variables, C and R, implicitly determine l_i. So, using a precomputed table of l_i values for a range of C and R values, we can proceed as before, using binary search to find the value of C such that $\sum_i l_i = P$. As before, we will exclude components with $\ln f_i(\mu_i) < C$ from calculations. As long as each adjacent component has a maximum log density greater than C, it will be possible to space them such that the log of their sum attains a minimum value of C.

Approximations to the entropy based on pairwise distances

In order to optimize the placement of the means to maximize entropy, we may minimize a proxy objective J based on pairwise distances between the components. We consider approximations to the entropy for both Gaussian and von Mises mixture models.

Gaussian Mixture Model with Rényi Entropy The Rényi 2-entropy, also known as collision entropy, is defined as $H_2(X) = -\ln \int p(x)^2 dx$. Maximizing this entropy is equivalent to minimizing the information potential $J = \int p(x)^2 dx$. For a mixture of Gaussians $p(x) = \sum_{i=1}^{N} \pi_i \mathcal{N}(x|\mu_i, \sigma^2)$, the integral of the squared density is:

$$J = \int \left(\sum_{i=1}^{N} \pi_i \mathcal{N}(x|\mu_i, \sigma^2) \right)^2 dx = \sum_{i=1}^{N} \sum_{j=1}^{N} \pi_i \pi_j \int \mathcal{N}(x|\mu_i, \sigma^2) \mathcal{N}(x|\mu_j, \sigma^2) dx$$

The integral of the product of two Gaussians is itself a Gaussian evaluated at the difference of their means:

$$\int \mathcal{N}(x|\mu_i, \sigma^2) \mathcal{N}(x|\mu_j, \sigma^2) dx = \mathcal{N}(0|\mu_i - \mu_j, 2\sigma^2) = \frac{1}{\sqrt{4\pi\sigma^2}} \exp\left(-\frac{(\mu_i - \mu_j)^2}{4\sigma^2} \right)$$

By dropping the leading scalar and the self-interaction terms ($i = j$) since they are constant with respect to the optimal placement of the means, the objective to minimize simplifies to the pairwise sum:

$$J = \sum_{i<j} \pi_i \pi_j \exp\left(-\frac{(\mu_i - \mu_j)^2}{4\sigma^2} \right)$$

Gaussian Mixture Model with Kolchinsky Objective Kolchinsky and Tracey [1] introduced a family of computationally efficient estimators for mixture entropy that rely on calculating pairwise distances between component distributions, proving they form a strict lower bound on the true mixture entropy. For homoskedastic Gaussian mixtures, maximizing this lower bound utilizing the Bhattacharyya distance yields a similar pairwise objective. The effective variance in the penalty term is simply doubled compared to the Rényi objective:

$$J = \sum_{i<j} \pi_i \pi_j \exp\left(-\frac{(\mu_i - \mu_j)^2}{8\sigma^2} \right)$$

Von Mises Mixture Model with Rényi Entropy When the components are distributed on the unit circle as von Mises distributions with homoskedastic concentration κ, we again aim to minimize the integral of the squared density. The integral of the cross-term of two von Mises components evaluates to a modified Bessel function of the first kind of order zero, I_0. Stripping out constants and self-terms, the objective becomes:

$$J = \sum_{i<j} \pi_i \pi_j I_0\left(2\kappa \cos\left(\frac{\mu_i - \mu_j}{2} \right) \right)$$

Von Mises Mixture Model with Kolchinsky Objective Applying Kolchinsky's lower bound using the Bhattacharyya coefficient to the von Mises mixture model yields an objective with an identical structure to the Rényi case. The difference lies solely in the concentration parameter, which is halved:

$$J = \sum_{i<j} \pi_i \pi_j I_0\left(\kappa \cos\left(\frac{\mu_i - \mu_j}{2} \right) \right)$$

An approximation for the entropy via Cramér's theorem

UNFINISHED Here we discuss a closed form approximation for the entropy in terms of the mixing weights, π_i, and the chosen means, μ_i. This enables efficient gradient descent on the means μ_i. We take a geometric approach and consider a large sample, $\mathbf{X}$, in N-dimensional space. Let A_i be a discrete random variable

taking on the values of μ_i, with respective probabilities π_i. Let $\mathcal{E}_i \sim \mathcal{N}(0, \sigma^2)$ be random noise on the ith component. Then we have that

$$\mathbf{X} = \mathbf{A} + \mathcal{E}$$

And

$$\begin{aligned} H(\mathbf{X}) &= H(\mathbf{A}, \mathbf{X}) - H(\mathbf{A}|\mathbf{X}) \\ &= H(\mathbf{A}, \mathcal{E}) - H(\mathbf{A}|\mathbf{X}) \quad \text{If } \mathbf{A} \text{ is fixed, } \mathbf{X} \text{ and } \mathcal{E} \text{ differ only be a translation} \\ &= H(\mathcal{E}) + H(\mathbf{A}) - H(\mathbf{A}|\mathbf{X}) \\ &= H(\mathcal{E}) + I(\mathbf{A}; \mathbf{X}) \end{aligned}$$

Since the entropy of the noise is constant, our objective is to maximize the mutual information between $\mathbf{X}$ and the component choice, $\mathbf{A}$. We frame this in terms of trying to infer the true component choice, $\mathbf{A}$, from the observed sample, $\mathbf{X}$. To this end, we consider the prior distribution of the vertices' distances to the sample, $D = \|\mathbf{X} - \mathbf{B}\|_2^2$, where $\mathbf{B}$ represents a candidate component choice and is distributed identically to $\mathbf{A}$. Noting that, $D = \sum_i D_i$ where $D_i = (A_i + \mathcal{E}_i - B_i)^2$, we have that

$$P(\bar{D} = d) \approx \exp\left(-NI(d)\right)$$

and the likelihood for $\bar{D}$ is

$$P(\mathbf{X}|\bar{D} = d) = \exp\left(-\frac{1}{2\sigma^2}Nd\right)$$

Thus the posterior mode of $\bar{D}$ is attained where $N(I(d) + \frac{d}{2\sigma^2})$ is minimized, which occurs when $I'(d) = -\frac{1}{2\sigma^2}$. At this point, the prior probability of the posterior mode will be $\exp(-NI(d))$, meaning that the observation delivers $NI(d)$ bits of information about the component choice, or $I(d)$ bits per sample.

$I(d)$ had a closed form expression, although its derivation is a somewhat formidable piece of algebra. We begin by defining the random variable $C_i = A_i - B_i$, which is also a discrete random variable taking on the values of $d_{ij} = \mu_i - \mu_j$ with probabilities $\rho_{ij} = \pi_i \pi_j$.

Solving for I in terms of its derivative, and setting it to $-\frac{1}{2\sigma^2}$, we have that

$$\mathbf{I}\left(\theta = -\frac{\mathbf{1}}{\mathbf{2\sigma^2}}\right) = \frac{\mathbf{1}}{\mathbf{2}}\ln(\mathbf{2}) - \frac{\mathbf{1}}{\mathbf{4}} - \ln(\mathbf{Z}) - \frac{\mathbf{1}}{\mathbf{8\sigma^2}}\frac{\mathbf{Y}}{\mathbf{Z}}$$

where

$$\mathbf{Z} = \sum_{\mathbf{i,j}} \rho_{\mathbf{ij}} \exp\left(-\frac{\mathbf{d_{ij}^2}}{\mathbf{4\sigma^2}}\right)$$

and

$$\mathbf{Y} = \sum_{\mathbf{i,j}} \rho_{\mathbf{ij}} \mathbf{d_{ij}^2} \exp\left(-\frac{\mathbf{d_{ij}^2}}{\mathbf{4\sigma^2}}\right)$$

.

The final term may be recognized as the expected squared distance of C_i under a Boltzmann distribution.

5.3 User Parameters

5.3.1 The Likelihood Model

We model our observations of the user's signal as being subject to two sources of noise: an outlier probability, ρ, that the signal was unintended, and additive noise on intended signals. We model the unintended signals as uniformly distributed, and the noise as a Gaussian. (We do not require that the noise have zero mean, since the user may be consistently late or early, relative to the intended time.) Hence our likelihood model for the received signal is a uniform-gaussian mixture model, characterized by the outlier probability, ρ, and the parameters of the noise, μ_ϵ, σ_ϵ.

5.3.2 Directional Statistics

Chapter 6

Render Stimulus

6.1 Circular Timers

In this step we represent the nodes' phases to the user. Dotter does this by drawing a small circular "timer" for each node.

We justify this choice with reference to the concept of "local attention".

One visually simpler choice would be to let a "selection" bar descend the screen from top to bottom, and to adjust nodes' vertical positions to reflect their phases. This approach resembles that row and column selection method of many communication boards.

Although this is less cluttered, it requires the use of global attention. By letting each node carry its own timer, the user does not need to be aware of any part of the screen outside of their foveal vision.

Chapter 7

Detect Response

Dotter requires the precise timing of the user's signal. Because the user's signal's variance is often as little a 20 milliseconds, and signal detection frames might be spaced as far apart as 30 milliseconds, the noise introduced by the signal detection process can be on the order of the noise introduced by the user.

7.1 Blink Detection

This frame frequency problem arises when the user's signal is a blink. A typical laptop webcam might capture 40 frames per second. If we use a simple threshold to detect a blink, this will introduce on the order of 20 milliseconds of noise. To resolve this problem we should use interpolation. The blink signal can be linearly interpolated between the frame before and the frame after the threshold was crossed to estimate the precise time of the blink. In post-hoc analysis making this change reduced about 3 milliseconds of noise when my blink standard deviation was about 20 milliseconds.

Further noise reductions should be made through a simple linear regression of features of the blink signal against the correct blink times. This can remove another millisecond of noise.

7.2 Keypress Detection

For an in-browser implementation, the `keydown` even should be preferred at the time should be read directly from the `event.timeStamp` field which provides the precise time that the keydown even was received by the browser.

Chapter 8

Determine Most Likely Break Nodes

One challenge of Dotter is that likelihoods are received with reference to a character-based trie, but priors are received with reference to a token-based trie. A considerable portion of the implementation is devoted to closing that gap.

In the chapter 3 we saw how to identify the character sequence prefixes with the highest posterior probabilities so that we could display them to the user. Here our task is to identify the token sequence prefixes with the highest posterior probabilities, so that we can query the language model for predictions. Just as we solved that problem with a character-based trie, so here we will solve it with a token-based trie. A token-based trie has a very high radix, equal to the model's vocabulary size, e.g., 50,257 for GPT-2.

8.1 An Upper Bound for the Likelihood of a Token Sequence

In the character-based trie (see chapter 2), calculating the likelihoods was trivial, but owing to the token-based nature of the language model, calculating the priors of a character sequence was somewhat involved. Here the situation is reversed. Calculating the prior of a token sequence is trivial, since that is directly what the language model returns, while calculating the likelihood of a token sequence prefix is difficult.

For any given token prefix A, and any token prefix sequence paritition set T (e.g. the set of all tokens, i.e., all token sequences of length 1), the likelihood of A is given by,

$$P(D|A) = \sum_{t \in T} P(D|A+t)P(A+t|A)$$

However, this poses a problem since we assume that $P(A+t|A)$ is not known. We are seeking to evaluate the likelihood of A in order to decide the priority of querying the language model for its next token, but in order to calculate this likelihood, we must already know the prior probabilities of the next token conditioned on A! Thus we are in a catch-22 and must resort to using a heuristic for the likelihood.

Since the likelihood of A is a weighted sum of its descendants' likelihoods, it must be bounded by the minimum and maximum of its descendants' likelihoods.

$$\min_{t \in T} P(D|A+t) \leq P(D|A) \leq \max_{t \in T} P(D|A+t)$$

Since we don't want to rely upon our current model of the prior, we require choose T so that all the descendants $A+t$ have a valid likelihood, i.e., so that they have never been a parent of a visible node.

We will prioritize token prefixes by their upper bound on the likelihood. Thus it is necessary to know, for a given node, the maximum likelihood of any descendant which has this node as a token ancestor. This is referred to as the `max_token_descendant_likelihood` of the node.

Maintaining this value during likelihood updates is discussed in chapter **??**.

8.2 Descending the Token Trie

Our choice of upper bound has the property that it strictly decreases with depth. This is because if a given node, x, is a token descendant of B, and B has A as a token ancestor, then x must be a token descendant of A as well and thus `A.max_token_descendant_likelihood` $\geq$ `B.max_token_descendant_likelihood`.

We make a priority-first search of the token trie, visiting the node with the highest upper bound on its posterior probability first. When we visit a node about which we have not yet queried the language model, we query the language model (chapter 9) about this node. The next token probabilities are referred to as the "LLM Result" which can then be consumed to update the character-based trie with a prior update (chapter **??**). Our descent of the token trie is not interrupted by the prior update, but we will need to restart our descent from the root after any likelihood update.

When we visit a node we must add its children to the priority queue. However, the radix is very high, equal to the vocabulary size (e.g., $V = 50,257$ for GPT-2). Thus we only add the children with the large posterior upper bounds to the queue. In implementation we use $K = 100$. To identify these one hundred children with the highest posterior upper bounds, we must calculate the posterior upper bound for all children. The language model itself gives us an array of length V for the prior probabilities of the children. Our task is to construct the array of likelihood upper bounds. To do so, we descend the character-based trie from the node in the character-based trie which corresponds to our current node in the token-based trie.

We descend the character-based trie until one of two conditions are met:

1. The descendant has a valid likelihood, i.e., it has never been a parent of a visible node. In this case, we know that this likelihood is the likelihood of all possible child token sequences. We set the relevant alphabetical range of the likelihood array to this likelihood. (E.g. if our node is 'the', and this descendant with a valid likelihood is 'the ho', we must set the likelihood array to this descendant's likelihood for every token in the range ' ho' to ' hp' such as the tokens ' horse', ' home', ' hound', etc.)
2. It is impossible for deeper descendants to be direct token children of our root node. I.e., the descendant does not extend our node by a strictly partial token prefix. In this case, we are at a direction token sequence child, 'B', of our root node, 'A'. For the specific element of the likelihood array corresponding to the last token of B, we set the likelihood to the maximum descendant likelihood of B which could have B as a token ancestor, i.e., `B.max_token_descendant_likelihood`.

Chapter 9

LLM Query

This step is straightforward. We call a model to find the distribution over the next token for a given token sequence. The only challenge is to correctly maintain the model's Key-Value Cache.

9.1 Maintaining a Key-Value Cache Trie

Traditionally, when we call a language model to generate a sequence of tokens, our Key-Value Cache is a simple list that grows as the sequence of tokens is extended. In our case, we must use a trie to store the Key-Value Cache.

Chapter 10

Mastering Dotter

10.1 The Cost of Incorrect Signals

Expertise in Dotter is about accuracy in three ways: (1) The user must synchronize their signals with the target timers. (2) The user must accurately select the correct prefix. (3) The user must not make involuntary signals (e.g. involuntary blinks).

From the program's perspective, there is virtually no difference between the user selecting the wrong prefix and making an involuntary signal. Both are considered as incorrect signals and both are assumed to have random phase. If the user intended the correct timer and was merely imprecise in synchronizing their signal, we do not consider this to be an "incorrect" signal. [1].

Here we will quantify the cost of incorrect signals.

UNFINISHED

The user's speed can be calculated from their performance parameters. Here we consider the effect of ρ, the probability of unintended signals. Given an observation, D, the gain in logits for the true hypothesis h is $\ln P(D|h) - \ln P(D|\neg h)$. When the posterior probability of h is small, this gain is equal to the gain in the log posterior probability of h. Let r denote the hypothesis that the signal was unintended, so that $P(r) = \rho$.

We state the following facts:

$$\begin{aligned} P(D|\neg h, r) = P(D|\neg h, \neg r) &= P(D|\neg h) \\ P(D|\neg h) &\approx P(D), \quad \text{when } P(h) \approx 0 \\ P(D|h, r) &= P(D), \quad \text{assuming that } D,\ h \text{ are independent conditioned on } r \end{aligned}$$

The likelihood under the true hypothesis is

$$P(D|h) = \rho P(D|h, r) + (1 - \rho) P(D|h, \neg r)$$

We consider the case where $P >> \sigma$ so that the components of the mixture are well-separated. This means that the contribution of the false component is negligible and that we can approximate the likelihood $P(D|h)$ piecewise as follows:

$$P(D|h) \approx \begin{cases} P(D|h, r)P(r) & \text{if } D \sim P(D|h, r) \\ P(D|h, \neg r)P(\neg r) & \text{if } D \sim P(D|h, \neg r). \end{cases}$$

That is, when D is likely drawn from the unintended distribution, $P(D|h) \approx P(D|h, r)P(r)$. When D is likely drawn from the intended distribution, $P(D|h) \approx P(D|h, \neg r)P(\neg r)$.

[1] We may safely assume that an involuntary signal has random phase. As for an incorrectly selected prefix, if we assume (perhaps unreasonably) that the incorrect prefix was randomly selected from all prefixes, and we further assume (legitimately) that we have uniformly distributed the prefixes' timers, then such a signal has random phase as well.

$$E_{D|r,h}\left[\ln P(D|h)\right] \approx E_{D|r,h}\left[\ln\Big(P(D|h,r)P(r)\Big)\right]$$
$$= \ln\rho - h(D|h,r)$$

and similarly

$$E_{D|\neg r,h}\left[\ln P(D|h)\right] \approx \ln(1-\rho) - h(D|h,\neg r)$$

The expected log likelihood of the data under the true hypothesis is

$$\begin{aligned}
E_D\left[\ln P(D|h)\right] &= \rho E_{D|r,h}\left[\ln P(D|h)\right] + (1-\rho)E_{D|\neg r,h}\left[\ln P(D|h)\right] \\
&\approx \rho(\ln\rho - h(D|h,r)) + (1-\rho)(\ln(1-\rho) - h(D|h,\neg r)) \\
&= -H_2(\rho) - \rho h(D|h,r) - (1-\rho)h(D|h,\neg r) \\
&= -H_2(\rho) - \rho h(D) - (1-\rho)h(D|h,\neg r), \quad \text{assuming } D|r,\ h|r, \text{ independent}
\end{aligned}$$

Where H_2 is the binary entropy function.

The likelihood of the alternative hypothesis is

$$E_{D|h}\left[\ln P(D|\neg h)\right] \approx E_{D|h}\left[\ln P(D)\right]$$

To simplify this term we use its expectation over h. Recalling the fact that,

$$E_a\left[E_{b|a}[f(b)]\right] = E_b[f(b)]$$

, we find that this term's expectation over h is $-h(D)$.

So the total expected gain in the logits of our hypothesis is

$$E_{D|h}[\ln P(D|h) - P(D|\neg h)] \approx -H_2(\rho) + (1-\rho)h(D) - (1-\rho)h(D|h,\neg r)$$

In our particular model, D is uniformly distributed over an interval of length P and the entropy of D conditional on h is just the entropy of the gaussian noise. Denoting the latter term by H_σ, we have,

$$E_{D|h}[\ln P(D|h) - P(D|\neg h)] \approx -H_2(\rho) + (1-\rho)(\ln P - H_\sigma)$$

This has a nice interpretation: we gain $\ln P - H_\sigma$ bits of information per intended signal, but the presence of unintended signals makes us pay a penalty in two ways: only $(1-\rho)$ of the time is our signal intended, and additionally we must expend $H_2(\rho)$ bits to learn whether the signal was intended.

Unfortunately the singularity of the binary entropy function at $\rho = 0$ becomes a practical problem for fast text entry. For example, a seemingly low outlier rate of $\rho = 0.03$ imposes the relatively high cost of $H_2(0.03) \approx 0.19$ bits in addition to the 3% loss that would be naively expected. Under reasonable parameters for an intermediate user (e.g. $\sigma = 25ms$, $P = 700ms$), these combined terms amount to a loss of more than 10% of the channel's capacity. Thus to achieve expert entry speeds, it is necessary to make unintended signals very rarely.

Bibliography

[1] Artemy Kolchinsky and Brendan Tracey. Estimating mixture entropy with pairwise distances. *Entropy*, 19(7):361, July 2017.

[2] David J. Ward, Alan F. Blackwell, and David J. C. MacKay. Dasher—a data entry interface using continuous gestures and language models. In *Proceedings of the 13th Annual ACM Symposium on User Interface Software and Technology*, UIST '00, pages 129–137, New York, NY, USA, 2000. ACM.